TRAVAUX PRATIQUES · DATA MINING

Analyser YouTube en temps réel

Guide illustré : les étapes, les outils et les techniques expliqués simplement, avec tout le code pas à pas



Python · MySQL · YouTube Data API · scikit-learn · Streamlit

Niveau Licence 3 / Master 1 · Durée : 4 heures

Document compagnon de GUIDE_TP.md et des scripts 01 à 09 + dashboard.py

Sommaire

1. Comprendre le projet en une page
2. Les outils utilisés et leur rôle
3. Les notions de base : API, JSON, quota, temps réel, base de données
4. Construire le projet pas à pas : tout le code expliqué
5. Les techniques de data mining expliquées simplement
6. Les variables : lesquelles, d'où, pourquoi
7. Éthique et protection des données
8. Problèmes fréquents et solutions
9. Glossaire

A. Annexe A — le simulateur (simulator.py)

Comment utiliser ce guide

Les parties 1 à 3 expliquent le projet, les outils et les notions de base. La **partie 4** contient **tout le code** du projet, fichier par fichier et bloc par bloc, avec la commande à lancer et le résultat attendu : l'étudiant construit le projet lui-même en la suivant.

Le fichier **GUIDE_TP.md** donne les consignes et les questions, et **EXERCICES_CORRIGES.md** les corrigés. Lisez ce PDF avant la séance, puis gardez-le ouvert pendant le TP.

Les graphiques présentés ont été produits avec le **mode DEMO** (données simulées). Avec une vraie clé YouTube, vous obtiendrez des graphiques du même type sur de vraies vidéos.

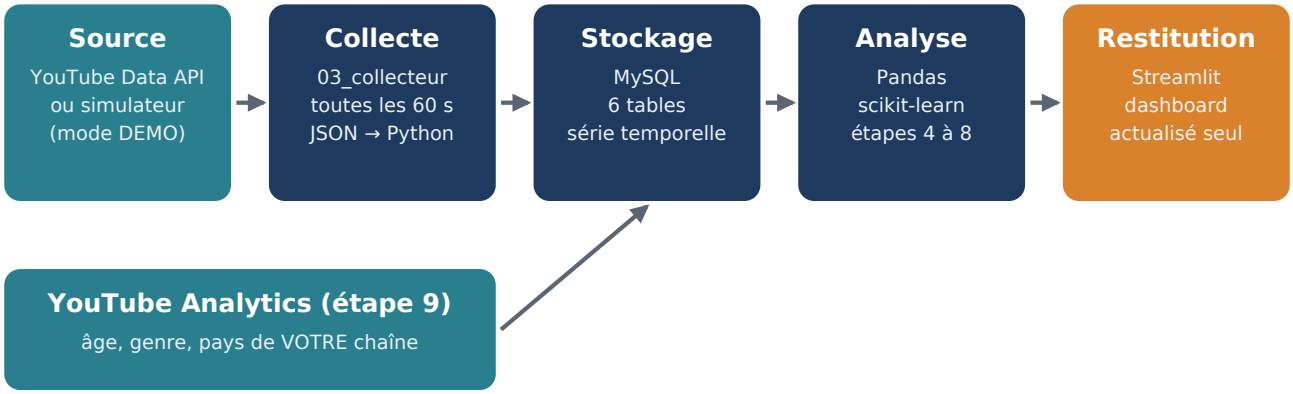
1. Comprendre le projet en une page

Le data mining (fouille de données) consiste à extraire des connaissances utiles à partir de grandes quantités de données : repérer des tendances, regrouper des éléments qui se ressemblent, prédire un résultat ou détecter ce qui sort de l'ordinaire.

Dans ce TP, on applique cette démarche à **YouTube**. On suit en direct une quinzaine de vidéos populaires au Congo, on enregistre leurs compteurs toutes les minutes, on récupère leurs commentaires, puis on répond à des questions concrètes :

- Quelles vidéos grandissent le plus vite, et à quel moment ?
- Les commentaires sont-ils plutôt positifs, négatifs ou neutres ?
- Peut-on regrouper les vidéos selon leur façon de devenir populaires ?
- Peut-on prédire le nombre de vues d'une vidéo ?
- Y a-t-il des hausses de vues suspectes ou des comptes qui spamment ?
- Quel est l'âge, le genre et le pays du public (pour sa propre chaîne) ?

Le pipeline : de YouTube jusqu'au tableau de bord



Architecture du projet : chaque bloc correspond à un ou plusieurs scripts.

Les 11 étapes en un coup d'œil

N°	Étape	Script	Ce qu'on obtient
0	Installation	—	Python, MySQL et les bibliothèques prêts
1	Test de l'API	01_test_api.py	La clé fonctionne, on voit le JSON
2	Choix des vidéos	02_choisir_videos.py	50 vidéos en base, 15 suivies
3	Collecte temps réel	03_collecteur_temps_reel.py	Relevés toutes les minutes, commentaires
4	Exploration	04_exploration.py	Statistiques, corrélations, graphiques
5	Sentiment	05_sentiment.py	Chaque commentaire classé +, − ou neutre
6	Clustering	06_clustering.py	Groupes de vidéos par profil de croissance
7	Prédiction	07_prediction.py	Modèle qui estime les vues
8	Anomalies	08_anomalies.py	Pics de vues et comptes suspects
9	Démographie	09_demographie.py	Âge, genre, pays (votre chaîne)
10	Dashboard	dashboard.py	Tableau de bord en direct

2. Les outils utilisés et leur rôle

Un projet de data mining fait intervenir plusieurs outils qui travaillent ensemble. Voici ce que fait chacun, et pourquoi on l'a choisi.

2.1 Les logiciels à installer

Outil	Rôle dans le TP	Pourquoi ce choix
Python 3.10+	Le langage dans lequel tous les scripts sont écrits.	Langage de référence en data science, gratuit, énorme écosystème.
VS Code	Éditeur pour lire et modifier le code, avec un terminal intégré.	Gratuit, léger, très bonne prise en charge de Python.
Environnement virtuel (venv)	Dossier `.venv` qui contient les bibliothèques du projet, isolées du reste de l'ordinateur.	Évite les conflits de versions entre projets.
pip	Installe les bibliothèques listées dans `requirements.txt`.	Gestionnaire de paquets standard de Python.
MySQL 8 (ou XAMPP / WAMP)	Serveur de base de données qui stocke toutes les données collectées.	Base relationnelle très répandue, enseignée en cours de SQL.
MySQL Workbench / phpMyAdmin	Interfaces graphiques pour voir les tables et lancer des requêtes SQL.	Permettent de vérifier visuellement ce que le collecteur enregistre.
SQLite (option)	Base de données dans un simple fichier, sans serveur.	Solution de secours si MySQL n'est pas installé.
Navigateur web	Affiche le dashboard Streamlit et la console Google Cloud.	—

2.2 Les services Google

Service	Rôle	Accès
Google Cloud Console	Site où l'on crée un projet et une clé API.	Gratuit avec un compte Gmail, sans carte bancaire.
YouTube Data API v3	Donne les données publiques : vidéos, chaînes, commentaires, tendances.	Clé API, 10 000 unités gratuites par jour.
YouTube Analytics API	Donne l'âge, le genre et le pays de l'audience **de sa propre chaîne**.	Connexion OAuth au compte propriétaire de la chaîne.

2.3 Les bibliothèques Python

Bibliothèque	À quoi elle sert	Utilisée dans
requests	Envoyer des requêtes HTTP à l'API YouTube et recevoir les réponses.	youtube_client.py
python-dotenv	Lire la configuration (clé, mot de passe) dans le fichier .env.	tous les scripts
SQLAlchemy	Dialoguer avec la base de données en Python, que ce soit MySQL ou SQLite.	db.py
PyMySQL	Pilote qui permet à SQLAlchemy de se connecter à MySQL.	db.py
pandas	Manipuler des tableaux de données (DataFrames) : filtrer, regrouper, calculer.	donnees.py, étapes 4 à 9
numpy	Calcul numérique (logarithmes, moyennes, tableaux).	étapes 6 à 8
scikit-learn	Les algorithmes de data mining : TF-IDF, régression logistique, K-Means, Random Forest, Isolation Forest, validation croisée.	étapes 5 à 8
joblib	Enregistrer le modèle de sentiment entraîné pour le réutiliser.	étape 5, collecteur
matplotlib	Créer les graphiques enregistrés dans figures/.	étapes 4, 6, 9
plotly	Graphiques interactifs du dashboard (zoom, survol).	dashboard.py
streamlit	Transformer un script Python en page web interactive.	dashboard.py
google-auth-oauthlib, **google-api-python-client**	Connexion sécurisée au compte Google pour YouTube Analytics.	09_demographie.py

2.4 Les fichiers « boîte à outils » du projet

Certains fichiers ne correspondent pas à une étape mais sont utilisés par toutes les étapes :

Fichier	Contenu
`.env`	La configuration : clé API, région, intervalle de collecte, accès à la base.
`db.py`	La connexion à la base et la définition des 6 tables. Crée la base automatiquement.
`youtube_client.py`	Les fonctions qui appellent l'API (vidéos, tendances, recherche, chaînes, commentaires) et transforment le JSON en dictionnaires Python. Contient aussi l'anonymisation.
`simulator.py`	Imite l'API YouTube hors ligne, avec des vidéos qui grandissent vraiment.
`pretraitement.py`	Le nettoyage des textes de commentaires.
`donnees.py`	Charge les tables en DataFrames et calcule les variables dérivées.

3. Les notions de base

3.1 Qu'est-ce qu'une API ?

Une **API** (interface de programmation) est une porte d'entrée qu'un service ouvre aux programmes. Au lieu d'ouvrir YouTube dans un navigateur et de lire les chiffres à l'écran, notre script envoie une **requête** à une adresse web précise, et YouTube répond avec les données demandées.

Analogie : l'API est comme le guichet d'une administration. On remplit un formulaire précis (la requête), on présente sa carte (la clé API), et on reçoit un document standardisé (la réponse JSON).

```
GET https://www.googleapis.com/youtube/v3/videos
      ?part=snippet,statistics,contentDetails
      &id=dQw4w9WgXcQ
      &key=VOTRE_CLE_API
```

3.2 Le format JSON

YouTube répond en **JSON**, un format texte organisé en paires « clé : valeur », imbriquées comme des dossiers. En Python, un JSON devient un **dictionnaire**. Exemple simplifié :

```
{
  "id": "demoVid021",
  "snippet": {
    "title": "Rumba congolaise — nouveau clip officiel",
    "publishedAt": "2026-10-25T08:29:31Z",
    "categoryId": "10"
  },
  "statistics": { "viewCount": "1791513", "likeCount": "91297", "commentCount": "14126" },
  "contentDetails": { "duration": "PT10M17S" }
}
```

- Les compteurs sont du **texte** (`"1791513"`) : il faut les convertir en nombres.
- La date est au format ISO, en **UTC** (le Z final).
- La durée `PT10M17S` signifie 10 minutes 17 secondes, soit 617 secondes.
- Ces conversions sont faites par `parse_video()` dans youtube_client.py.

3.3 La clé API et le quota

La clé API identifie votre projet. Google limite chaque projet à **10 000 unités par jour** (remise à zéro à minuit, heure du Pacifique, soit 8 h ou 9 h à Brazzaville selon la saison). Chaque type de requête a un coût :

Requête	Ce qu'elle renvoie	Coût
videos.list	Statistiques et infos de 1 à 50 vidéos	1 unité
videos.list (chart=mostPopular)	Les vidéos en tendance d'un pays	1 unité
channels.list	Infos de 1 à 50 chaînes	1 unité
commentThreads.list	Jusqu'à 100 commentaires d'une vidéo	1 unité
search.list	Résultats d'une recherche par mots-clés	**100 unités**

Avec les réglages par défaut (relevé toutes les 60 s, commentaires tous les 10 cycles, 15 vidéos), le collecteur consomme environ **3 000 unités en 24 heures**.

> **Attention**
> Ne publiez jamais votre clé API (dans un rapport, sur GitHub, sur WhatsApp). Quelqu'un pourrait épuiser votre quota. Elle reste dans le fichier .env.

3.4 « Temps réel » : collecte périodique ou streaming ?

Collecte périodique (ce TP)



Streaming continu



chaque événement arrive dès qu'il se produit (chat d'un live, flux Bluesky...)

YouTube Data API ne propose pas de flux continu pour les vues : on fait donc une **collecte périodique**. À chaque relevé, on enregistre l'état des compteurs avec l'heure exacte. Entre deux relevés, on calcule la **vitesse** (nouvelles vues par minute). C'est du « quasi temps réel », suffisant pour voir une vidéo décoller en direct.

3.5 La base de données : 6 tables

Toutes les données arrivent dans une base relationnelle. Chaque table a un rôle précis ; elles sont reliées par des identifiants (`video_id`, `channel_id`, `author_hash`).

Table	Une ligne =	Colonnes principales	Remplie par
videos	une vidéo	titre, tags, catégorie, date, durée, suivie (oui/non), cluster	étape 2, étape 6
video_stats	un relevé d'une vidéo	date du relevé, vues, likes, commentaires	étapes 2 et 3
channels	une chaîne	nom, pays, abonnés, nombre de vidéos	étapes 2 et 3
comments	un commentaire	texte, likes, date, auteur anonymisé, sentiment, suspect	étapes 3, 5, 8
commenters	un commentateur (anonyme)	ancienneté du compte, pays déclaré, abonnés, vidéos	étape 3
demographics	une mesure d'audience	tranche d'âge, genre, pays, pourcentage	étape 9

À retenir

La table **video_stats** est le cœur du « temps réel » : elle grandit de 15 lignes à chaque minute. Après 2 heures de collecte, elle contient environ 1 800 relevés, c'est-à-dire une vraie série temporelle.

3.6 L'anonymisation par hash

Un **hash** est une empreinte : une fonction mathématique transforme un texte en une suite de caractères de longueur fixe. Le même texte donne toujours la même empreinte, mais on ne peut pas revenir en arrière.

```
identifiant YouTube :  UCx8a3...kQ          (jamais stocké)
+ sel secret        :  "change_moi..."
          ↓ SHA-256
author_hash         :  9f2c41d07be3a5e0c1...  (stocké dans la base)
```

On peut ainsi compter combien de commentaires a écrit un même auteur, **sans savoir qui il est**. Le **sel** (texte secret ajouté avant le calcul) empêche quelqu'un de recalculer les empreintes de tous les identifiants connus pour retrouver les auteurs.

4. Construire le projet pas à pas : tout le code

Dans cette partie, vous allez écrire **tout le projet vous-même**, fichier par fichier, dans l'ordre où il faut les créer. Chaque fichier est découpé en **blocs** : une explication, puis le code correspondant. À la fin de chaque étape : la commande à lancer et le résultat que vous devez obtenir.

4.0 La méthode de travail

- Créez un dossier `TP_YouTube_DataMining` et ouvrez-le dans VS Code (**Fichier → Ouvrir le dossier**).
- Pour chaque fichier : **Fichier → Nouveau fichier**, enregistrez-le avec le nom exact indiqué dans le bandeau bleu, puis tapez ou copiez les blocs **dans l'ordre, les uns à la suite des autres**, dans ce même fichier.
- Lisez l'explication **avant** chaque bloc : c'est elle qui vous fait comprendre le code.
- Lancez la commande de fin d'étape et comparez avec le résultat attendu avant de continuer.

> **Attention**
>
> En Python, **l'indentation (les espaces en début de ligne) fait partie du code**. Un bloc mal indenté provoque une `IndentationError`. On utilise 4 espaces par niveau.
>
> Le copier-coller depuis un PDF peut abîmer l'indentation ou certains caractères selon le lecteur PDF. En cas de doute, comparez avec les fichiers de l'archive `TP_YouTube_DataMining.zip`, qui contiennent exactement le même code.
>
> Taper le code soi-même (au moins pour les étapes 3, 5 et 6) est la meilleure façon de le comprendre.

4.1 L'arborescence finale du projet

```
TP_YouTube_DataMining/
├── requirements.txt            ← étape 0 : bibliothèques
├── .env.example  →  .env       ← étape 0 : configuration
├── schema.sql                  ← étape 0 : base MySQL
├── db.py                       ← étape 0 : connexion et tables
├── youtube_client.py           ← étape 1 : dialogue avec YouTube
├── simulator.py                ← étape 1 : simulateur (annexe A)
├── 01_test_api.py              ← étape 1
├── 02_choisir_videos.py        ← étape 2
├── pretraitement.py            ← étape 3 : nettoyage des textes
├── 03_collecteur_temps_reel.py ← étape 3
├── donnees.py                  ← étape 4 : chargement + variables
├── 04_exploration.py           ← étape 4
├── data/corpus_sentiment.csv   ← étape 5
├── 05_sentiment.py             ← étape 5
├── 06_clustering.py            ← étape 6
├── 07_prediction.py            ← étape 7
├── 08_anomalies.py             ← étape 8
├── 09_demographie.py           ← étape 9
├── dashboard.py                ← étape 10
├── models/   figures/          ← créés automatiquement par les scripts
└── .venv/                      ← créé par python -m venv
```

4.2 Quels fichiers dépendent de quels autres ?

Fichier	Il importe…	Il est utilisé par…
db.py	—	tous les scripts
youtube_client.py	simulator.py (en mode DEMO)	01, 02, 03
pretraitement.py	—	03 (collecteur), 05 (sentiment)
donnees.py	db.py, youtube_client.py	04 à 08, dashboard.py

C'est pour cela que l'ordre de création compte : un script ne peut pas fonctionner si un fichier qu'il importe n'existe pas encore (erreur `ModuleNotFoundError`).

0 Préparer le projet et la base

`python db.py` — 30 min

Objectif : Créer le dossier du projet, installer les bibliothèques, configurer la connexion et créer la base de données.

Fichiers à créer : `requirements.txt`, `.env.example`, `schema.sql`, `db.py`

Fichier `requirements.txt` — nouveau fichier

La liste des bibliothèques Python du projet. La commande `pip install -r requirements.txt` les installe toutes en une fois. Le symbole >= signifie « cette version ou une plus récente ».

Le fichier complet

Les deux dernières bibliothèques ne servent qu'à l'étape 9 (démographie). Les lignes qui commencent par # sont des commentaires.

```
requests>=2.31
python-dotenv>=1.0
pandas>=2.0
numpy>=1.24
SQLAlchemy>=2.0
PyMySQL>=1.1
cryptography>=41
scikit-learn>=1.3
joblib>=1.3
matplotlib>=3.7
plotly>=5.18
streamlit>=1.37
# Étape 8 (démographie, optionnelle) :
google-auth-oauthlib>=1.2
google-api-python-client>=2.100
```

Fichier `.env.example` nouveau fichier

Le modèle de configuration. On le copie en `.env` (sans « .example ») puis on remplit ses propres valeurs. Ainsi, les mots de passe et la clé API ne sont jamais écrits dans le code.

Le fichier complet

Chaque ligne a la forme `NOM=valeur`, sans espace autour du signe =. Au début, laissez `YT_API_KEY=DEMO` : tout fonctionnera avec des données simulées. Pensez à changer `MYSQL_PASSWORD` et `ANON_SALT`.

```
# ===== Clé YouTube Data API v3 =====
# Mettre DEMO pour travailler hors ligne avec des données simulées.
YT_API_KEY=DEMO

# Pays pour les tendances (CG = Congo-Brazzaville, CD = RDC, FR = France, CI = Côte d'Ivoire...)
REGION_CODE=CG

# Nombre de vidéos suivies en temps réel
TRACK_TOP=15

# ===== Collecte =====
# Secondes entre deux relevés des compteurs
COLLECT_INTERVAL_SECONDS=60
# Les commentaires sont relevés tous les N cycles (économise le quota)
COMMENTS_EVERY_N_CYCLES=10

# ===== Base de données =====
# mysql (recommandé) ou sqlite (aucune installation nécessaire)
DB_ENGINE=mysql
MYSQL_HOST=127.0.0.1
MYSQL_PORT=3306
MYSQL_USER=root
MYSQL_PASSWORD=ton_mot_de_passe
MYSQL_DATABASE=datamining_youtube
SQLITE_PATH=datamining_youtube.db

# Sel secret pour anonymiser les auteurs de commentaires (changer cette valeur !)
ANON_SALT=change_moi_avec_un_texte_secret
```

Fichier `schema.sql` nouveau fichier

Le script SQL qui crée la base `datamining_youtube` et ses 6 tables. Il est facultatif (db.py crée aussi les tables), mais il permet de bien voir la structure de la base et de la créer depuis MySQL Workbench ou phpMyAdmin.

Le script complet

`utf8mb4` permet de stocker tous les caractères, y compris les émojis des commentaires. `PRIMARY KEY` garantit qu'une vidéo ou un commentaire n'existe qu'une seule fois. Les `INDEX` accélèrent les recherches par vidéo et par date, très fréquentes dans le TP.

```
-- schema.sql — Base MySQL du TP Data Mining YouTube
-- Exécution : mysql -u root -p < schema.sql   (ou copier-coller dans MySQL Workbench)
-- Remarque : les scripts Python créent aussi les tables automatiquement.

CREATE DATABASE IF NOT EXISTS datamining_youtube
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE datamining_youtube;

CREATE TABLE IF NOT EXISTS channels (
  channel_id        VARCHAR(64) PRIMARY KEY,
  title             VARCHAR(255),
  country           VARCHAR(8),
  published_at      DATETIME,
  subscriber_count  BIGINT,
```

```
  video_count       BIGINT,
  view_count        BIGINT,
  updated_at        DATETIME
);

CREATE TABLE IF NOT EXISTS videos (
  video_id          VARCHAR(32) PRIMARY KEY,
  channel_id        VARCHAR(64),
  title             TEXT,
  description       TEXT,
  tags              TEXT,
  category_id       VARCHAR(8),
  default_language  VARCHAR(16),
  published_at      DATETIME,
  duration_seconds  INT,
  source            VARCHAR(20),
  tracked           INT DEFAULT 0,
  first_seen        DATETIME,
  cluster           INT
);

-- Table "temps réel" : un relevé des compteurs par vidéo et par cycle
CREATE TABLE IF NOT EXISTS video_stats (
  id             INT AUTO_INCREMENT PRIMARY KEY,
  video_id       VARCHAR(32),
  collected_at   DATETIME,
  view_count     BIGINT,
  like_count     BIGINT,
  comment_count  BIGINT,
  INDEX idx_stats_video (video_id),
  INDEX idx_stats_time (collected_at)
);

CREATE TABLE IF NOT EXISTS comments (
  comment_id       VARCHAR(128) PRIMARY KEY,
  video_id         VARCHAR(32),
  author_hash      VARCHAR(64),   -- auteur anonymisé (hash), jamais le nom
  text             TEXT,
  like_count       INT,
  reply_count      INT,
  published_at     DATETIME,
  collected_at     DATETIME,
  sentiment        VARCHAR(10),
  sentiment_score  FLOAT,
  is_suspect       INT,
  INDEX idx_com_video (video_id),
  INDEX idx_com_author (author_hash)
);

CREATE TABLE IF NOT EXISTS commenters (
  author_hash         VARCHAR(64) PRIMARY KEY,
  channel_created_at  DATETIME,
  country             VARCHAR(8),
  subscriber_count    BIGINT,
  video_count         BIGINT,
  updated_at          DATETIME
```

```
);

CREATE TABLE IF NOT EXISTS demographics (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  dimension   VARCHAR(20),
  age_group   VARCHAR(20),
  gender      VARCHAR(20),
  country     VARCHAR(8),
  metric      VARCHAR(40),
  value       FLOAT,
  start_date  VARCHAR(10),
  end_date    VARCHAR(10)
);
```

Fichier `db.py` nouveau fichier

La connexion à la base de données et la description des tables en Python. Tous les autres scripts l'importent. Il utilise **SQLAlchemy**, qui permet d'écrire le même code pour MySQL et pour SQLite.

Bloc 1/7 — Imports et configuration

Le texte entre triple guillemets est une **docstring** : une description du fichier. `load_dotenv()` lit le fichier .env et rend ses valeurs disponibles avec `os.getenv()`. `MetaData()` est le « catalogue » qui va contenir la description de toutes nos tables.

```
"""
db.py — Connexion à la base de données et définition des tables.

Fonctionne avec MySQL (DB_ENGINE=mysql) ou SQLite (DB_ENGINE=sqlite).
Les tables sont créées automatiquement au premier lancement.
"""

import os
from datetime import datetime, timezone
from urllib.parse import quote_plus

from dotenv import load_dotenv
from sqlalchemy import (
    BigInteger,
    Column,
    DateTime,
    Float,
    Integer,
    MetaData,
    String,
    Table,
    Text,
    create_engine,
    select,
    text,
)

load_dotenv()

metadata = MetaData()
```

Bloc 2/7 — Les tables channels et videos

Chaque `Table(...)` décrit une table : son nom, puis ses colonnes avec leur type (`String` = texte court, `Text` = texte long, `BigInteger` = grand entier pour les vues, `DateTime` = date et heure). `primary_key=True` désigne l'identifiant unique. La colonne `tracked` vaut 1 pour les vidéos suivies en temps réel.

```
# Une ligne par chaîne YouTube (créateurs des vidéos suivies)
channels = Table(
    "channels",
    metadata,
    Column("channel_id", String(64), primary_key=True),
    Column("title", String(255)),
    Column("country", String(8)),
    Column("published_at", DateTime),
    Column("subscriber_count", BigInteger),
    Column("video_count", BigInteger),
    Column("view_count", BigInteger),
    Column("updated_at", DateTime),
    mysql_charset="utf8mb4",
)
```

```
# Une ligne par vidéo (informations fixes)
videos = Table(
    "videos",
    metadata,
    Column("video_id", String(32), primary_key=True),
    Column("channel_id", String(64)),
    Column("title", Text),
    Column("description", Text),
    Column("tags", Text),
    Column("category_id", String(8)),
    Column("default_language", String(16)),
    Column("published_at", DateTime),
    Column("duration_seconds", Integer),
    Column("source", String(20)),  # trending / search / manual
    Column("tracked", Integer, default=0),  # 1 = suivie en temps réel
    Column("first_seen", DateTime),
    Column("cluster", Integer),  # rempli à l'étape 6
    mysql_charset="utf8mb4",
)
```

Bloc 3/7 — Les tables video_stats et comments

video_stats est la table du temps réel : une nouvelle ligne par vidéo à chaque relevé. Son identifiant `id` est un compteur automatique (`autoincrement`). `index=True` crée un index pour retrouver vite les relevés d'une vidéo. Dans **comments**, l'auteur n'est stocké que sous forme de hash (`author_hash`).

```
# Une ligne par relevé (c'est la table "temps réel")
video_stats = Table(
    "video_stats",
    metadata,
    Column("id", Integer, primary_key=True, autoincrement=True),
    Column("video_id", String(32), index=True),
    Column("collected_at", DateTime, index=True),
    Column("view_count", BigInteger),
    Column("like_count", BigInteger),
    Column("comment_count", BigInteger),
    mysql_charset="utf8mb4",
)

# Une ligne par commentaire (auteur anonymisé)
```

```
comments = Table(
    "comments",
    metadata,
    Column("comment_id", String(128), primary_key=True),
    Column("video_id", String(32), index=True),
    Column("author_hash", String(64), index=True),
    Column("text", Text),
    Column("like_count", Integer),
    Column("reply_count", Integer),
    Column("published_at", DateTime),
    Column("collected_at", DateTime),
    Column("sentiment", String(10)),  # rempli à l'étape 5 / par le collecteur
    Column("sentiment_score", Float),
    Column("is_suspect", Integer),  # rempli à l'étape 8
    mysql_charset="utf8mb4",
)
```

Bloc 4/7 — Les tables commenters et demographics

commenters garde le profil public et anonymisé des auteurs (ancienneté du compte, pays déclaré, abonnés). **demographics** reçoit les pourcentages d'âge, de genre et de pays de l'étape 9.

```
# Profil PUBLIC et anonymisé des commentateurs
commenters = Table(
    "commenters",
    metadata,
    Column("author_hash", String(64), primary_key=True),
    Column("channel_created_at", DateTime),
    Column("country", String(8)),
    Column("subscriber_count", BigInteger),
    Column("video_count", BigInteger),
    Column("updated_at", DateTime),
    mysql_charset="utf8mb4",
)

# Démographie de TA chaîne (étape 8, YouTube Analytics)
```

```
demographics = Table(
    "demographics",
    metadata,
    Column("id", Integer, primary_key=True, autoincrement=True),
    Column("dimension", String(20)),  # age_gender / country
    Column("age_group", String(20)),
    Column("gender", String(20)),
    Column("country", String(8)),
    Column("metric", String(40)),
    Column("value", Float),
    Column("start_date", String(10)),
    Column("end_date", String(10)),
    mysql_charset="utf8mb4",
)
```

Bloc 5/7 — L'heure UTC et l'adresse de la base

`utcnow()` renvoie l'heure universelle : toutes les dates de la base sont en UTC pour éviter les confusions de fuseau. `get_url()` construit l'adresse de connexion à partir du .env. `quote_plus` protège les caractères spéciaux du mot de passe (@, #, :).

```
def utcnow() -> datetime:
    """Heure actuelle en UTC (sans fuseau, comme dans la base)."""
    return datetime.now(timezone.utc).replace(tzinfo=None, microsecond=0)


def get_url() -> str:
    engine = os.getenv("DB_ENGINE", "mysql").lower()
    if engine == "sqlite":
        return f"sqlite:///{os.getenv('SQLITE_PATH', 'datamining_youtube.db')}"
    user = os.getenv("MYSQL_USER", "root")
    pwd = quote_plus(os.getenv("MYSQL_PASSWORD", ""))
    host = os.getenv("MYSQL_HOST", "127.0.0.1")
    port = os.getenv("MYSQL_PORT", "3306")
    name = os.getenv("MYSQL_DATABASE", "datamining_youtube")
    return f"mysql+pymysql://{user}:{pwd}@{host}:{port}/{name}?charset=utf8mb4"
```

■ Bloc 6/7 — La création automatique de la base

`_creer_base_mysql()` se connecte au serveur et exécute `CREATE DATABASE IF NOT EXISTS`. `get_engine()` crée la connexion une seule fois (variable `global _engine`) puis `metadata.create_all()` crée les tables qui n'existent pas encore.

```
_engine = None


def _creer_base_mysql():
    """Crée la base MySQL si elle n'existe pas encore."""
    url = get_url()
    serveur, nom = url.rsplit("/", 1)[0], os.getenv("MYSQL_DATABASE", "datamining_youtube")
    eng = create_engine(serveur + "/?charset=utf8mb4", future=True)
    with eng.begin() as conn:
        conn.execute(
            text(
                f"CREATE DATABASE IF NOT EXISTS `{nom}` "
                "CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci"
            )
```

```
        )
    eng.dispose()


def get_engine():
    """Retourne le moteur SQLAlchemy et crée la base et les tables si besoin."""
    global _engine
    if _engine is None:
        if os.getenv("DB_ENGINE", "mysql").lower() != "sqlite":
            _creer_base_mysql()
        _engine = create_engine(get_url(), pool_pre_ping=True, future=True)
        metadata.create_all(_engine)
    return _engine
```

■ Bloc 7/7 — Insérer ou mettre à jour

« Upsert » = **up**date + in**sert**. Si la clé existe déjà, on met à jour la ligne ; sinon, on l'insère. C'est ce qui évite les doublons de vidéos et de chaînes. Le bloc `if __name__ == "__main__":` ne s'exécute que si l'on lance `python db.py` directement : il sert de test de connexion.

```
def upsert(conn, table, key_col: str, row: dict):
    """Insère la ligne, ou la met à jour si la clé existe déjà
    (fonctionne de la même façon sur MySQL et SQLite)."""
    key = row[key_col]
    exists = conn.execute(select(table.c[key_col]).where(table.c[key_col] == key)).first()
    if exists:
        values = {k: v for k, v in row.items() if k != key_col and v is not None}
        if values:
            conn.execute(table.update().where(table.c[key_col] == key).values(**values))
        return False
    conn.execute(table.insert().values(**row))
    return True


if __name__ == "__main__":
    eng = get_engine()
    print("Connexion OK :", eng.url.render_as_string(hide_password=True))
    print("Tables :", ", ".join(metadata.tables.keys()))
```

Une fois les quatre fichiers créés, installez l'environnement :

```
python -m venv .venv              # crée l'environnement virtuel
.venv\Scripts\activate            # l'active (Linux : source .venv/bin/activate)
pip install -r requirements.txt   # installe les bibliothèques
copy .env.example .env            # crée la configuration (Linux : cp)
```

Ouvrez `.env` et renseignez `MYSQL_PASSWORD` (ou mettez `DB_ENGINE=sqlite` si MySQL n'est pas installé). Créez la base, au choix avec `mysql -u root -p < schema.sql` ou simplement

en lançant db.py.

Lancer

```
python db.py
```

Résultat attendu (avec SQLite : Connexion OK : sqlite:///datamining_youtube.db)

```
Connexion OK : mysql+pymysql://root:***@127.0.0.1:3306/datamining_youtube?charset=utf8mb4
Tables : channels, videos, video_stats, comments, commenters, demographics
```

1 Le client YouTube et le test de l'API

`python 01_test_api.py` — 30 min

Objectif : Écrire le code qui dialogue avec YouTube, puis vérifier que la clé fonctionne.

Fichiers à créer : `youtube_client.py`, `simulator.py` (annexe A), `01_test_api.py`

Créer la clé API (5 minutes, gratuit)

1. **console.cloud.google.com** → se connecter avec un compte Gmail.
2. **Nouveau projet** → nom : tp-datamining.
3. **API et services → Bibliothèque** → YouTube Data API v3 → **Activer**.
4. **Identifiants → Créer des identifiants → Clé API**.
5. Copier la clé (AIza...) dans YT_API_KEY du fichier .env. Sans clé, laissez DEMO.

Fichier `youtube_client.py` — nouveau fichier

Le « traducteur » entre notre programme et YouTube. Il envoie les requêtes à l'API, compte le quota, gère les erreurs et transforme le JSON reçu en dictionnaires Python simples. Avec YT_API_KEY=DEMO, il interroge le simulateur à la place de YouTube.

Bloc 1/9 — Imports et adresse de l'API

`requests` envoie les requêtes HTTP, `hashlib` calcule les empreintes pour l'anonymisation, `re` sert aux expressions régulières. `API_URL` est l'adresse commune à toutes les requêtes.

```
"""
youtube_client.py — Petit client pour la YouTube Data API v3.

- Avec une vraie clé : interroge https://www.googleapis.com/youtube/v3
- Avec YT_API_KEY=DEMO : utilise simulator.py (données simulées, hors ligne)

Coût en quota (10 000 unités gratuites par jour) :
  videos.list ......... 1 unité pour 50 vidéos
  channels.list ....... 1 unité pour 50 chaînes
  commentThreads.list . 1 unité pour 100 commentaires
  search.list ......... 100 unités (à utiliser avec modération !)
"""

import hashlib
```

```
import os
import re
from datetime import datetime, timezone

import requests
from dotenv import load_dotenv

load_dotenv()

API_URL = "https://www.googleapis.com/youtube/v3"
```

■ Bloc 2/9 — Les catégories et l'erreur personnalisée

YouTube ne donne qu'un numéro de catégorie : ce **dictionnaire** le traduit en nom lisible. `class YouTubeError(Exception)` crée notre propre type d'erreur, pour pouvoir distinguer une erreur de l'API d'un autre problème.

```
# Catégories standard de YouTube
CATEGORIES = {
    "1": "Film & Animation",
    "2": "Autos",
    "10": "Musique",
    "15": "Animaux",
    "17": "Sport",
    "19": "Voyages",
    "20": "Jeux vidéo",
    "22": "Blogs & Personnes",
    "23": "Humour",
    "24": "Divertissement",
    "25": "Actualités & Politique",
    "26": "Vie pratique & Style",
    "27": "Éducation",
    "28": "Science & Technologie",
}


class YouTubeError(Exception):
    pass
```

■ Bloc 3/9 — La classe et l'appel générique _get

Une **classe** regroupe des données (la clé, le quota utilisé) et des fonctions (les méthodes). `__init__` est appelée à la création de l'objet. `_get()` est le seul endroit qui parle vraiment à YouTube : il ajoute la clé, envoie la requête, vérifie le code de réponse (200 = succès) et renvoie le JSON. En mode DEMO, il appelle `simulator.handle()` à la place.

```
class YouTubeClient:
    def __init__(self, api_key: str | None = None):
        self.api_key = api_key or os.getenv("YT_API_KEY", "DEMO")
        self.demo = self.api_key.strip().upper() == "DEMO"
        self.quota_used = 0
        if self.demo:
            import simulator

            self._sim = simulator

    # ---------- appel générique ----------
    def _get(self, endpoint: str, params: dict, cost: int = 1) -> dict:
        self.quota_used += cost
        if self.demo:
```

```
            return self._sim.handle(endpoint, dict(params))
        params = dict(params, key=self.api_key)
        r = requests.get(f"{API_URL}/{endpoint}", params=params, timeout=30)
        if r.status_code != 200:
            try:
                err = r.json()["error"]
                reason = err.get("errors", [{}])[0].get("reason", "")
                msg = err.get("message", "")
            except Exception:
                reason, msg = "", r.text[:200]
            raise YouTubeError(f"{r.status_code} {reason} : {msg}")
        return r.json()
```

Bloc 4/9 — Vidéos, tendances et recherche

`videos()` découpe la liste en paquets de 50 (la limite de l'API) avec `range(0, len(...), 50)`. `trending()` utilise `chart=mostPopular` pour obtenir les tendances d'un pays. `search()` coûte 100 unités : on l'indique avec `cost=100`.

```
    # ---------- vidéos ----------
    def videos(self, video_ids: list[str]) -> list[dict]:
        out = []
        for i in range(0, len(video_ids), 50):
            data = self._get(
                "videos",
                {
                    "part": "snippet,statistics,contentDetails",
                    "id": ",".join(video_ids[i : i + 50]),
                    "maxResults": 50,
                },
            )
            out += [parse_video(it) for it in data.get("items", [])]
        return out
```

```
    def trending(self, region: str = "CG", max_results: int = 50) -> list[dict]:
        data = self._get(
            "videos",
            {
                "part": "snippet,statistics,contentDetails",
                "chart": "mostPopular",
                "regionCode": region,
                "maxResults": min(max_results, 50),
            },
        )
        return [parse_video(it) for it in data.get("items", [])]

    def search(
        self,
        query: str,
        region: str | None = None,
        max_results: int = 25,
        order: str = "viewCount",
    ) -> list[str]:
        """Retourne des identifiants de vidéos. ATTENTION : 100 unités !"""
        params = {
            "part": "id",
            "q": query,
            "type": "video",
            "maxResults": min(max_results, 50),
            "order": order,
        }
        if region:
            params["regionCode"] = region
        data = self._get("search", params, cost=100)
        return [
            it["id"]["videoId"]
            for it in data.get("items", [])
            if it.get("id", {}).get("videoId")
        ]
```

Bloc 5/9 — Chaînes et commentaires

`dict.fromkeys(...)` supprime les doublons en gardant l'ordre. Pour les commentaires, `order="time"` demande les plus récents. Si une vidéo a ses commentaires désactivés, l'API répond par une erreur 403 : on renvoie alors une liste vide au lieu d'arrêter le programme.

```
    # ---------- chaînes ----------
    def channels(self, channel_ids: list[str]) -> list[dict]:
        out = []
        ids = list(dict.fromkeys(c for c in channel_ids if c))
        for i in range(0, len(ids), 50):
            data = self._get(
                "channels",
                {
                    "part": "snippet,statistics",
                    "id": ",".join(ids[i : i + 50]),
                    "maxResults": 50,
                },
            )
            out += [parse_channel(it) for it in data.get("items", [])]
```

```
        return out

    # ---------- commentaires ----------
    def comments(
        self, video_id: str, max_results: int = 100, order: str = "time"
    ) -> list[dict]:
        try:
            data = self._get(
                "commentThreads",
                {
                    "part": "snippet",
                    "videoId": video_id,
                    "maxResults": min(max_results, 100),
                    "order": order,
                    "textFormat": "plainText",
                },
            )
        except YouTubeError as e:
            if "commentsDisabled" in str(e) or "403" in str(e):
                return []  # commentaires désactivés sur cette vidéo
            raise
        return [parse_comment(it, video_id) for it in data.get("items", [])]
```

Bloc 6/9 — Fonctions de conversion

`parse_dt()` convertit une date texte ISO en objet `datetime` UTC. `parse_duration()` lit une durée comme PT1H2M30S avec une expression régulière et la convertit en secondes. `_int()` convertit un compteur texte en nombre, ou renvoie None s'il est absent (likes masqués).

```
# ================== Transformation JSON -> dictionnaires ==================


def parse_dt(s: str | None):
    if not s:
        return None
    s = s.replace("Z", "+00:00")
    try:
        return datetime.fromisoformat(s).astimezone(timezone.utc).replace(tzinfo=None)
    except ValueError:
        return None


def parse_duration(iso: str | None) -> int | None:
```

```
    """'PT1H2M30S' -> 3750 secondes"""
    if not iso:
        return None
    m = re.match(r"P(?:(\d+)D)?T?(?:(\d+)H)?(?:(\d+)M)?(?:(\d+)S)?", iso)
    if not m:
        return None
    d, h, mi, s = (int(x) if x else 0 for x in m.groups())
    return d * 86400 + h * 3600 + mi * 60 + s


def _int(x):
    return int(x) if x not in (None, "") else None
```

Bloc 7/9 — Transformer une vidéo

On extrait les trois parties du JSON (`snippet`, `statistics`, `contentDetails`) et on renvoie un dictionnaire « à plat » avec des noms de colonnes identiques à ceux de la base. `.get(clé, défaut)` évite une erreur si une clé manque. Les tags sont réunis en un seul texte séparé par `|`.

```
def parse_video(item: dict) -> dict:
    sn, st, cd = (
        item.get("snippet", {}),
        item.get("statistics", {}),
        item.get("contentDetails", {}),
    )
    return {
        "video_id": item["id"],
        "channel_id": sn.get("channelId"),
        "title": sn.get("title"),
        "description": (sn.get("description") or "")[:5000],
        "tags": "|".join(sn.get("tags", [])),
        "category_id": sn.get("categoryId"),
        "default_language": sn.get("defaultAudioLanguage") or sn.get("defaultLanguage"),
        "published_at": parse_dt(sn.get("publishedAt")),
        "duration_seconds": parse_duration(cd.get("duration")),
        # compteurs (likes peuvent être masqués -> None)
        "view_count": _int(st.get("viewCount")),
        "like_count": _int(st.get("likeCount")),
        "comment_count": _int(st.get("commentCount")),
    }
```

Bloc 8/9 — Transformer une chaîne et anonymiser

Certaines chaînes masquent leur nombre d'abonnés : on stocke alors `None`. `anonymize()` ajoute le sel secret à l'identifiant, calcule l'empreinte SHA-256 et garde les 32 premiers caractères : on ne peut pas retrouver l'identifiant d'origine.

```
def parse_channel(item: dict) -> dict:
    sn, st = item.get("snippet", {}), item.get("statistics", {})
    return {
        "channel_id": item["id"],
        "title": sn.get("title"),
        "country": sn.get("country"),
        "published_at": parse_dt(sn.get("publishedAt")),
        "subscriber_count": (
            None if st.get("hiddenSubscriberCount") else _int(st.get("subscriberCount"))
        ),
        "video_count": _int(st.get("videoCount")),
        "view_count": _int(st.get("viewCount")),
    }


def anonymize(author_channel_id: str | None) -> str | None:
    """Remplace l'identifiant de l'auteur par un hash irréversible."""
    if not author_channel_id:
        return None
    salt = os.getenv("ANON_SALT", "tp-datamining")
    return hashlib.sha256((salt + author_channel_id).encode()).hexdigest()[:32]
```

Bloc 9/9 — Transformer un commentaire

L'identifiant de l'auteur est gardé **uniquement en mémoire** (`author_channel_id`), le temps de récupérer son profil public. Seul `author_hash` sera enregistré dans la base.

```
def parse_comment(item: dict, video_id: str) -> dict:
    top = item["snippet"]["topLevelComment"]
    sn = top["snippet"]
    author_id = (sn.get("authorChannelId") or {}).get("value")
    return {
        "comment_id": top["id"],
        "video_id": video_id,
        "author_channel_id": author_id,  # utilisé UNIQUEMENT en mémoire, jamais stocké
        "author_hash": anonymize(author_id),
        "text": sn.get("textOriginal") or sn.get("textDisplay") or "",
        "like_count": _int(sn.get("likeCount")) or 0,
        "reply_count": _int(item["snippet"].get("totalReplyCount")) or 0,
        "published_at": parse_dt(sn.get("publishedAt")),
    }
```

Fichier `01_test_api.py` — nouveau fichier

Le premier script du TP. Il vérifie que la clé fonctionne et montre la différence entre la réponse JSON brute de YouTube et la même vidéo transformée en dictionnaire Python.

Bloc 1/4 — Préparation

On importe notre client, puis on crée l'objet `client`. `os.getenv("REGION_CODE", "CG")` lit le pays dans le .env, avec « CG » comme valeur par défaut. L'expression `A if condition else B` choisit le texte à afficher selon le mode.

```
"""
ÉTAPE 1 — Tester la clé API et découvrir le format JSON.

Lancement :  python 01_test_api.py
"""

import json
import os

from youtube_client import CATEGORIES, YouTubeClient, YouTubeError

client = YouTubeClient()
region = os.getenv("REGION_CODE", "CG")
print("Mode :", "DÉMO (données simulées)" if client.demo else "RÉEL (API YouTube)")
```

Bloc 2/4 — Appel protégé par try / except

Le bloc `try` tente l'appel ; si une `YouTubeError` survient (clé invalide, pas d'Internet), le bloc `except` affiche un message clair et `raise SystemExit(1)` arrête le script proprement. On vérifie ensuite qu'au moins une vidéo a été renvoyée.

```
try:
    # Réponse brute (JSON) d'une vidéo en tendance
    brut = client._get(
        "videos",
        {
            "part": "snippet,statistics,contentDetails",
            "chart": "mostPopular",
            "regionCode": region,
            "maxResults": 1,
        },
    )
except YouTubeError as e:
    print("\n[ERREUR] Erreur de l'API :", e)
    print(
```

```
        "Vérifie : la clé dans .env, que 'YouTube Data API v3' est activée, "
        "et ta connexion Internet."
    )
    raise SystemExit(1)

if not brut.get("items"):
    print(f"\n[ATTENTION] Aucune tendance disponible pour REGION_CODE={region}.")
    print("Essaie une autre région dans .env (ex. FR, CD, CI, CM) puis relance.")
    raise SystemExit(1)
```

Bloc 3/4 — Afficher le JSON brut

`json.dumps(..., indent=2)` affiche le JSON joliment indenté. `ensure_ascii=False` garde les accents lisibles. La partie entre accolades `{k: ... for k in (...)}` est une **compréhension de dictionnaire** : elle ne garde que quatre champs du snippet.

```
print("\n===== Extrait de la réponse JSON brute =====")
item = brut["items"][0]
print(
    json.dumps(
        {
            "id": item["id"],
            "snippet": {
                k: item["snippet"][k]
                for k in ("title", "channelId", "publishedAt", "categoryId")
            },
            "statistics": item["statistics"],
            "contentDetails": item["contentDetails"],
        },
        indent=2,
        ensure_ascii=False,
    )
)
```

Bloc 4/4 — Afficher la version transformée

`client.trending(region, 1)[0]` renvoie directement la vidéo convertie. Dans `f"{k:18}"`, le `:18` aligne les noms sur 18 caractères pour un affichage en colonnes.

```
v = client.trending(region, 1)[0]
print("\n===== Après transformation (dictionnaire Python) =====")
for k in (
    "video_id",
    "title",
    "published_at",
    "duration_seconds",
    "category_id",
    "view_count",
    "like_count",
    "comment_count",
):
    print(f"{k:18}: {v[k]}")
print("Catégorie          :", CATEGORIES.get(v["category_id"], "?"))
print(
    f"\n[OK] API OK — quota utilisé par ce script : {client.quota_used} unité(s) "
    "sur 10 000/jour"
)
```

> **Le simulateur**
>
> Avant de lancer le script en mode DEMO, créez aussi `simulator.py` : son code complet est en **annexe A**. Avec une vraie clé, il n'est pas utilisé.

Lancer

```
python 01_test_api.py
```

Résultat attendu (mode DEMO)

```
Mode : DÉMO (données simulées)

===== Extrait de la réponse JSON brute =====
{
  "id": "demoVid021",
  "snippet": {
    "title": "Comment réussir son bac en 3 mois #21",
    "channelId": "demoChan02",
    "publishedAt": "2026-10-25T08:55:11Z",
    "categoryId": "23"
  },
  "statistics": {
    "viewCount": "1791513",
    "likeCount": "91297",
    "commentCount": "14126"
  },
  "contentDetails": {
    "duration": "PT10M17S"
  }
}

===== Après transformation (dictionnaire Python) =====
video_id          : demoVid021
title             : Comment réussir son bac en 3 mois #21
published_at      : 2026-10-25 08:55:11
duration_seconds  : 617
category_id       : 23
view_count        : 1791513
like_count        : 91297
comment_count     : 14126
Catégorie         : Humour

[OK] API OK — quota utilisé par ce script : 2 unité(s) sur 10 000/jour
```

2 Choisir les vidéos

`python 02_choisir_videos.py` — 10 min

Objectif : Constituer l'échantillon : 50 vidéos en base, dont 15 suivies en temps réel.

Fichiers à créer : `02_choisir_videos.py`

Fichier `02_choisir_videos.py` — nouveau fichier

Choisit les vidéos à étudier (tendances, recherche ou liste précise), les enregistre en base avec leur chaîne et un premier relevé, et marque les plus vues comme « suivies ».

Bloc 1/5 — Documentation et imports

La docstring rappelle les différentes façons de lancer le script. On importe les tables et fonctions de `db.py` et notre client YouTube.

```
"""
ÉTAPE 2 — Choisir les vidéos à étudier et les enregistrer en base.

Exemples :
  python 02_choisir_videos.py                          # tendances du pays (REGION_CODE)
  python 02_choisir_videos.py --region FR              # tendances d'un autre pays
  python 02_choisir_videos.py --search "rumba congolaise"   # recherche (coûte 100 unités)
  python 02_choisir_videos.py --ids abc123,def456      # vidéos précises (ID de l'URL)

Toutes les vidéos trouvées sont enregistrées (utiles pour l'étape 7),
mais seules les TRACK_TOP premières sont suivies en temps réel.
"""

import argparse
import os
from sqlalchemy import update

from db import channels, get_engine, upsert, utcnow, video_stats, videos
from youtube_client import YouTubeClient
```

Bloc 2/5 — Les options de la ligne de commande

argparse lit les options tapées après le nom du script (`--region`, `--search`…). Chaque `add_argument` déclare une option, son type et sa valeur par défaut. Après `parse_args()`, on y accède avec `args.region`, `args.search`, etc.

```
parser = argparse.ArgumentParser()
parser.add_argument("--region", default=os.getenv("REGION_CODE", "CG"))
parser.add_argument("--search", help="mots-clés à rechercher")
parser.add_argument("--ids", help="identifiants de vidéos séparés par des virgules")
parser.add_argument(
    "--track",
    type=int,
    default=int(os.getenv("TRACK_TOP", 15)),
    help="nombre de vidéos à suivre en temps réel",
)
args = parser.parse_args()

client = YouTubeClient()
now = utcnow()
```

Bloc 3/5 — Récupérer les vidéos

Selon l'option choisie, on récupère une liste précise, le résultat d'une recherche ou les tendances. `found.sort(key=lambda v: -(...))` trie par vues décroissantes (le signe moins inverse l'ordre). On récupère ensuite les chaînes de toutes ces vidéos en un seul appel.

```
if args.ids:
    source = "manual"
    found = client.videos([x.strip() for x in args.ids.split(",") if x.strip()])
elif args.search:
    source = "search"
    found = client.videos(client.search(args.search, region=args.region, max_results=50))
else:
    source = "trending"
    found = client.trending(args.region, 50)

if not found:
    print(f"Aucune vidéo trouvée (région {args.region}). Essaie --region FR ou --search.")
    raise SystemExit(1)

# Les plus vues d'abord
found.sort(key=lambda v: -(v["view_count"] or 0))
chan_info = {c["channel_id"]: c for c in client.channels([v["channel_id"] for v in found])}
```

■ Bloc 4/5 — Enregistrer en base

`with eng.begin() as conn:` ouvre une **transaction** : si une erreur survient, rien n'est enregistré à moitié. On remet d'abord `tracked` à 0 partout (la nouvelle sélection remplace l'ancienne). `v.pop(k)` retire les compteurs du dictionnaire de la vidéo pour les écrire dans `video_stats`. `enumerate` donne le rang : les `args.track` premières vidéos sont suivies.

```
eng = get_engine()
with eng.begin() as conn:
    # la nouvelle sélection remplace l'ancienne pour le suivi temps réel
    conn.execute(update(videos).values(tracked=0))
    for rank, v in enumerate(found):
        stats = {k: v.pop(k) for k in ("view_count", "like_count", "comment_count")}
        v.update(source=source, first_seen=now, tracked=1 if rank < args.track else 0)
        upsert(conn, videos, "video_id", v)
        # premier relevé
        conn.execute(
            video_stats.insert().values(video_id=v["video_id"], collected_at=now, **stats)
        )
    for c in chan_info.values():
        upsert(conn, channels, "channel_id", dict(c, updated_at=now))
```

■ Bloc 5/5 — Afficher le résumé

On affiche le nombre de vidéos enregistrées et la liste des vidéos suivies. `found[: args.track]` prend les premiers éléments de la liste (découpage, ou *slicing*).

```
print(
    f"{len(found)} vidéos enregistrées ({source}), "
    f"{min(args.track, len(found))} suivies en temps réel :\n"
)
for v in found[: args.track]:
    print(f"  - {v['video_id']}  {v['title'][:70]}")
print(
    f"\nQuota utilisé : {client.quota_used} unité(s). "
    "Étape suivante : python 03_collecteur_temps_reel.py"
)
```

Lancer

```
python 02_choisir_videos.py
```

Résultat attendu (mode DEMO)

```
40 vidéos enregistrées (trending), 15 suivies en temps réel :

  - demoVid021  Comment réussir son bac en 3 mois #21
  - demoVid007  Débat : la jeunesse et le chômage #7
  - demoVid005  Micro-trottoir au marché Total #5
  - demoVid030  Les meilleurs buts de la semaine #30
  - demoVid033  Débat : la jeunesse et le chômage #33
  - demoVid029  Interview exclusive d'un artiste #29
  - demoVid025  Prank au quartier Bacongo #25
  - demoVid000  Mariage coutumier à Brazzaville #0
  - demoVid006  Ndombolo 2026 : la danse qui enflamme Brazzaville #6
  - demoVid037  Débat : la jeunesse et le chômage #37
  - demoVid002  Concert live au Palais des congrès #2
  - demoVid003  Les meilleurs buts de la semaine #3
  - demoVid032  Diables Rouges : résumé du match #32
  - demoVid035  Mariage coutumier à Brazzaville #35
  - demoVid009  Vlog : une journée à Oyo #9

Quota utilisé : 2 unité(s). Étape suivante : python 03_collecteur_temps_reel.py
```

3 Le collecteur en temps réel

`python 03_collecteur_temps_reel.py` — 20 min

Objectif : Enregistrer en continu l'évolution des vidéos et leurs commentaires.

Fichiers à créer : `pretraitement.py`, `03_collecteur_temps_reel.py`

Fichier `pretraitement.py` — nouveau fichier

Le nettoyage des textes de commentaires et le chargement du modèle de sentiment. Ce fichier sert à l'étape 5 (entraînement) et au collecteur (prédiction en direct). On le crée dès maintenant car le collecteur l'importe.

Bloc 1/3 — Imports et emplacement du modèle

`MODEL_PATH` construit le chemin `models/sentiment.joblib` à partir du dossier du fichier, pour que le chemin fonctionne d'où que l'on lance le script.

```
"""
pretraitement.py — Nettoyage des textes de commentaires (utilisé par
l'étape 5 pour entraîner le modèle et par le collecteur pour prédire
le sentiment en temps réel).
"""

import os
import re
import unicodedata

import joblib

MODEL_PATH = os.path.join(
    os.path.dirname(os.path.abspath(__file__)), "models", "sentiment.joblib"
)
```

Bloc 2/3 — Nettoyer un texte

Chaque `re.sub(motif, remplacement, texte)` remplace ce qui correspond au motif. Étapes : minuscules, liens remplacés par le mot « lien », suppression des chiffres, suppression des accents (`unicodedata` sépare la lettre de son accent, puis on retire l'accent), suppression de la ponctuation sauf ! et ?, et enfin des espaces multiples.

```
def nettoyer(texte: str) -> str:
    """minuscules, sans accents, sans liens, sans ponctuation inutile."""
    if not isinstance(texte, str):
        return ""
    t = texte.lower()
    t = re.sub(r"https?://\S+|www\.\S+", " lien ", t)  # les liens deviennent le mot "lien"
    t = re.sub(r"\d+", " ", t)
    t = unicodedata.normalize("NFD", t)
    t = "".join(c for c in t if unicodedata.category(c) != "Mn")  # retire les accents
    t = re.sub(r"[^a-z!? ]+", " ", t)  # garde ! et ? (utiles au sentiment)
    t = re.sub(r"(!|\?)", r" \1 ", t)
    return re.sub(r"\s+", " ", t).strip()
```

Bloc 3/3 — Charger le modèle et prédire

`charger_modele()` renvoie None tant que l'étape 5 n'a pas été faite : le collecteur sait alors qu'il ne doit pas prédire. `predire()` nettoie les textes, prédit l'étiquette et renvoie la probabilité de la classe choisie comme score de confiance.

```
def charger_modele():
    """Retourne le modèle entraîné, ou None s'il n'existe pas encore."""
    if os.path.exists(MODEL_PATH):
        return joblib.load(MODEL_PATH)
    return None


def predire(modele, textes: list[str]):
    """Retourne (étiquettes, score de confiance)."""
    propres = [nettoyer(t) for t in textes]
    labels = modele.predict(propres)
    scores = modele.predict_proba(propres).max(axis=1)
    return list(labels), [float(s) for s in scores]
```

Fichier `03_collecteur_temps_reel.py` nouveau fichier

Le cœur du TP. Une boucle sans fin qui, toutes les minutes, relève les compteurs des vidéos suivies et, régulièrement, récupère les nouveaux commentaires, prédit leur sentiment et enregistre le profil anonymisé de leurs auteurs.

Bloc 1/9 — Documentation et imports

La docstring résume les 5 tâches du collecteur. On importe toutes les tables utiles, le nettoyage/prédiction et le client YouTube.

```
"""
ÉTAPE 3 — Collecteur en (quasi) temps réel.

À chaque cycle (toutes les COLLECT_INTERVAL_SECONDS secondes) :
  1. relève vues / likes / commentaires des vidéos suivies  -> table video_stats
Tous les COMMENTS_EVERY_N_CYCLES cycles :
  2. récupère les nouveaux commentaires (dédoublonnés)       -> table comments
  3. prédit leur sentiment si le modèle de l'étape 5 existe
  4. récupère le profil PUBLIC des nouveaux auteurs (anonymisé) -> table commenters
  5. met à jour les statistiques des chaînes                 -> table channels

Lancement :  python 03_collecteur_temps_reel.py
Options   :  --cycles 5 (s'arrête après 5 cycles)   --interval 30
Arrêt     :  Ctrl + C
"""

import argparse
import os
import time

from sqlalchemy import select

from db import channels, commenters, comments, get_engine, upsert, utcnow, video_stats, videos
from pretraitement import charger_modele, predire
from youtube_client import YouTubeClient, YouTubeError, anonymize
```

Bloc 2/9 — Les options

`--interval` (secondes entre deux relevés), `--comments-every` (fréquence des commentaires) et `--cycles` (nombre de cycles avant arrêt, 0 = sans fin, pratique pour tester). Les valeurs par défaut viennent du .env.

```
parser = argparse.ArgumentParser()
parser.add_argument(
    "--interval", type=int, default=int(os.getenv("COLLECT_INTERVAL_SECONDS", 60))
)
parser.add_argument(
    "--comments-every", type=int, default=int(os.getenv("COMMENTS_EVERY_N_CYCLES", 10))
)
parser.add_argument("--cycles", type=int, default=0, help="0 = sans fin")
args = parser.parse_args()
```

Bloc 3/9 — Lire la liste des vidéos suivies

`select(videos.c.video_id).where(videos.c.tracked == 1)` est l'équivalent SQLAlchemy de `SELECT video_id FROM videos WHERE tracked = 1`. `known_authors` est un **ensemble** (set) des auteurs déjà connus : on ne redemandera pas leur profil. Sans vidéo suivie, on arrête avec un message.

```
client = YouTubeClient()
eng = get_engine()

with eng.connect() as conn:
    tracked = [
        r[0] for r in conn.execute(select(videos.c.video_id).where(videos.c.tracked == 1))
    ]
    known_authors = {r[0] for r in conn.execute(select(commenters.c.author_hash))}
if not tracked:
    print("Aucune vidéo suivie. Lance d'abord : python 02_choisir_videos.py")
    raise SystemExit(1)
```

Bloc 4/9 — Estimer le quota et charger le modèle

On calcule combien d'unités le collecteur consommerait en 24 heures avec ces réglages, et on prévient l'étudiant si l'on dépasse 9 000. Puis on charge le modèle de sentiment s'il existe.

```
# Estimation du quota journalier pour prévenir l'étudiant
cycles_jour = 86400 / args.interval
estimation = cycles_jour * (len(tracked) / 50 + 0.0001) + cycles_jour / args.comments_every * (
    len(tracked) + 2
)
print(
    f"Mode {'DÉMO' if client.demo else 'RÉEL'} — {len(tracked)} vidéos suivies, "
    f"relevé toutes les {args.interval}s"
)
print(f"Quota estimé si le collecteur tourne 24 h : ~{int(estimation)} unités / 10 000")
if estimation > 9000:
    conseil = "augmente --interval ou --comments-every."
    print("[ATTENTION] Risque de dépasser le quota :", conseil)

modele = charger_modele()
print(
    "Sentiment en temps réel :",
    "activé" if modele else "désactivé (lance l'étape 5 pour l'activer)",
)
print("Ctrl + C pour arrêter.\n")
```

Bloc 5/9 — La boucle principale

`while True:` répète indéfiniment. Le `try` extérieur attrape `KeyboardInterrupt` (Ctrl + C) pour arrêter proprement. Le `try` intérieur protège chaque cycle : une erreur n'arrête pas tout le collecteur.

```
cycle = 0
try:
    while True:
        cycle += 1
        now = utcnow()
        try:
```

Bloc 6/9 — Tâche 1 : relever les compteurs

Un seul appel récupère les compteurs de toutes les vidéos suivies. Pour chacune, on insère une ligne dans `video_stats` avec l'heure du relevé. C'est cette table qui formera la série temporelle. Dans `{total_vues:,}`, la virgule ajoute un séparateur de milliers.

```
            # ---------- 1. compteurs ----------
            items = client.videos(tracked)
            with eng.begin() as conn:
                for v in items:
                    conn.execute(
                        video_stats.insert().values(
                            video_id=v["video_id"],
                            collected_at=now,
                            view_count=v["view_count"],
                            like_count=v["like_count"],
                            comment_count=v["comment_count"],
                        )
                    )
            total_vues = sum(v["view_count"] or 0 for v in items)
            msg = (
                f"[{now:%H:%M:%S}] cycle {cycle:>4} | {len(items)} vidéos "
                f"| {total_vues:,} vues au total"
            )
```

Bloc 7/9 — Tâche 2 : les nouveaux commentaires

`cycle % args.comments_every == 0` est vrai un cycle sur N (l'opérateur `%` donne le reste de la division). Pour chaque commentaire, on vérifie s'il est déjà en base : si oui, `continue` passe au suivant (c'est le **dédoublonnage**). `c.pop("author_channel_id")` retire l'identifiant réel du dictionnaire : il ne sera jamais enregistré.

```
            # ---------- 2 à 5. commentaires, profils, chaînes ----------
            if cycle == 1 or cycle % args.comments_every == 0:
                nouveaux, nouveaux_auteurs = [], {}
                with eng.connect() as conn:
                    for vid in tracked:
                        for c in client.comments(vid, max_results=100, order="time"):
                            if conn.execute(
                                select(comments.c.comment_id).where(
                                    comments.c.comment_id == c["comment_id"]
                                )
                            ).first():
                                continue
                            author_id = c.pop("author_channel_id")
                            if (
                                c["author_hash"]
                                and c["author_hash"] not in known_authors
                                and author_id
                            ):
                                nouveaux_auteurs[author_id] = c["author_hash"]
                            nouveaux.append(c)
```

Bloc 8/9 — Tâches 3 à 5 : sentiment, profils, chaînes

Si le modèle existe, on prédit le sentiment de tous les nouveaux commentaires d'un coup. On récupère ensuite le profil public des nouveaux auteurs et les statistiques des chaînes, puis on enregistre tout dans une même transaction. **c « déplie » le dictionnaire : chaque clé devient une colonne.

```
                if nouveaux and modele is not None:
                    labels, scores = predire(modele, [c["text"] for c in nouveaux])
                    for c, lab, sc in zip(nouveaux, labels, scores):
                        c["sentiment"], c["sentiment_score"] = lab, sc
                # profils publics des nouveaux auteurs (on ne garde que le hash)
                profils = client.channels(list(nouveaux_auteurs)) if nouveaux_auteurs else []
                chans = client.channels(list({v["channel_id"] for v in items}))
                with eng.begin() as conn:
                    for c in nouveaux:
                        conn.execute(comments.insert().values(collected_at=now, **c))
                    for p in profils:
                        h = nouveaux_auteurs.get(p["channel_id"]) or anonymize(p["channel_id"])
                        upsert(
                            conn,
                            commenters,
                            "author_hash",
                            {
                                "author_hash": h,
                                "channel_created_at": p["published_at"],
                                "country": p["country"],
                                "subscriber_count": p["subscriber_count"],
                                "video_count": p["video_count"],
                                "updated_at": now,
                            },
                        )
                        known_authors.add(h)
                    for ch in chans:
                        upsert(conn, channels, "channel_id", dict(ch, updated_at=now))
                msg += f" | +{len(nouveaux)} commentaires | +{len(profils)} profils"
            print(msg + f" | quota utilisé : {client.quota_used}")
```

Bloc 9/9 — Gestion des erreurs et pause

Une erreur d'API est affichée ; si le quota est épuisé, on sort de la boucle avec `break`. Toute autre erreur (coupure réseau) est affichée et on réessaie au cycle suivant. `time.sleep(args.interval)` met le programme en pause entre deux cycles.

```
        except YouTubeError as e:
            print(f"[{now:%H:%M:%S}] Erreur API : {e}")
            if "quotaExceeded" in str(e):
                print(
                    "Quota journalier épuisé. "
                    "Reprise possible demain (minuit heure du Pacifique)."
                )
                break
        except Exception as e:  # coupure réseau, base indisponible...
            print(f"[{now:%H:%M:%S}] Erreur : {e} — nouvel essai au prochain cycle")

        if args.cycles and cycle >= args.cycles:
            break
        time.sleep(args.interval)
except KeyboardInterrupt:
    pass
print(
    f"\nCollecte arrêtée après {cycle} cycle(s). "
    f"Quota utilisé : {client.quota_used} unités."
)
```

Lancer

```
python 03_collecteur_temps_reel.py
```

Résultat attendu (mode DEMO)

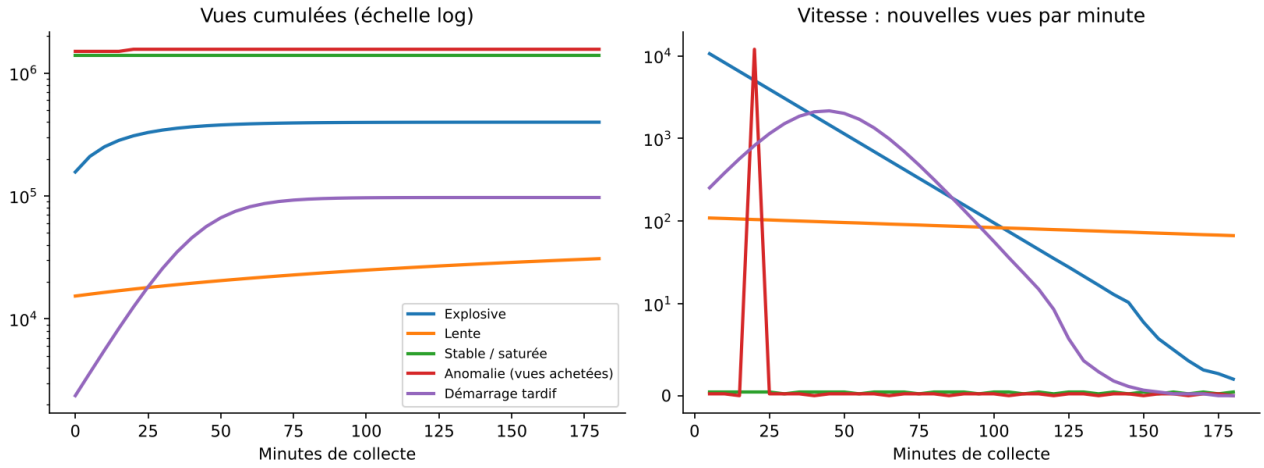
```
Mode DÉMO — 15 vidéos suivies, relevé toutes les 60s
Quota estimé si le collecteur tourne 24 h : ~2880 unités / 10 000
Sentiment en temps réel : désactivé (lance l'étape 5 pour l'activer)
Ctrl + C pour arrêter.

[08:40:05] cycle    1 | 15 vidéos | 9,662,722 vues au total | +1500 commentaires | +395 profils | quota utilis…
[08:40:11] cycle    2 | 15 vidéos | 9,665,212 vues au total | quota utilisé : 26
[08:40:17] cycle    3 | 15 vidéos | 9,667,464 vues au total | quota utilisé : 27
[08:40:23] cycle    4 | 15 vidéos | 9,669,710 vues au total | +56 commentaires | +0 profils | quota utilisé : 44
[08:40:30] cycle    5 | 15 vidéos | 9,672,016 vues au total | quota utilisé : 45
[08:40:36] cycle    6 | 15 vidéos | 9,674,245 vues au total | quota utilisé : 46
[08:40:42] cycle    7 | 15 vidéos | 9,676,466 vues au total | quota utilisé : 47
[08:40:48] cycle    8 | 15 vidéos | 9,678,682 vues au total | +72 commentaires | +1 profils | quota utilisé : 65
…
```

À retenir

Laissez ce terminal ouvert : le collecteur tourne en continu. Ouvrez un **second terminal** (avec `.venv` activé) pour les étapes suivantes.

Pour un test rapide : `python 03_collecteur_temps_reel.py --interval 10 --cycles 5`.



Ce que le collecteur permet d'observer : cinq façons de devenir populaire (mode DEMO, 3 heures). Le pic rouge est une hausse brutale de vues, retrouvée à l'étape 8.

Questions

1. Pourquoi ajoute-t-on un « sel » secret avant de calculer le hash d'un auteur ?
2. Dans MySQL, exécutez `SELECT video_id, collected_at, view_count FROM video_stats ORDER BY collected_at DESC LIMIT 20;`. Que constatez-vous ?
3. Pourquoi les commentaires sont-ils relevés moins souvent que les compteurs ?

4 Analyse exploratoire

`python 04_exploration.py` — 30 min

Objectif : Charger les données dans pandas, calculer les variables et explorer.

Fichiers à créer : `donnees.py`, `04_exploration.py`

Fichier `donnees.py` — nouveau fichier

Le pont entre la base et l'analyse. Il charge les tables dans des **DataFrames** pandas et calcule les variables dérivées (engagement, vitesse, âge...). Toutes les étapes d'analyse et le dashboard l'utilisent.

Bloc 1/5 — Lire une table

`pd.read_sql_table("videos", engine)` charge toute une table dans un DataFrame, c'est-à-dire un tableau avec des colonnes nommées.

```
"""
donnees.py — Chargement des tables en DataFrames Pandas + variables calculées.
Utilisé par les étapes 4 à 8 et par le dashboard.
"""

import numpy as np
import pandas as pd

from db import get_engine, utcnow
from youtube_client import CATEGORIES


def lire(table: str) -> pd.DataFrame:
    return pd.read_sql_table(table, get_engine())
```

Bloc 2/5 — La série temporelle

On charge `video_stats`, on convertit la colonne date avec `pd.to_datetime` et on trie par vidéo puis par date : indispensable pour calculer les différences entre relevés successifs.

```
def releves() -> pd.DataFrame:
    """Tous les relevés (série temporelle), triés par vidéo et par date."""
    s = lire("video_stats")
    if s.empty:
        return s
    s["collected_at"] = pd.to_datetime(s["collected_at"])
    return s.sort_values(["video_id", "collected_at"])
```

Bloc 3/5 — Une ligne par vidéo

`groupby("video_id").tail(1)` garde le **dernier relevé** de chaque vidéo. `join` et `merge` ajoutent ces compteurs et les informations de la chaîne (comme une jointure SQL). `how="left"` garde toutes les vidéos même sans chaîne connue.

```
def table_videos() -> pd.DataFrame:
    """Une ligne par vidéo : infos fixes + dernier relevé + chaîne + variables."""
    v = lire("videos")
    if v.empty:
        return v
    s = releves()
    dernier = (
        s.groupby("video_id")
        .tail(1)
        .set_index("video_id")[["view_count", "like_count", "comment_count", "collected_at"]]
    )
    v = v.join(dernier, on="video_id")
    ch = lire("channels").rename(
        columns={
```

```
            "title": "channel_title",
            "country": "channel_country",
            "published_at": "channel_created_at",
            "view_count": "channel_view_count",
            "video_count": "channel_video_count",
        }
    )
    v = v.merge(
        ch[
            [
                "channel_id",
                "channel_title",
                "channel_country",
                "subscriber_count",
                "channel_video_count",
                "channel_view_count",
            ]
        ],
        on="channel_id",
        how="left",
    )
```

Bloc 4/5 — Les variables calculées

Chaque ligne crée une nouvelle colonne pour toutes les vidéos d'un coup (calcul **vectorisé**, sans boucle). `.dt.hour` extrait l'heure d'une date. `replace(0, np.nan)` évite les divisions par zéro : diviser par `NaN` donne `NaN` (valeur manquante) au lieu d'une erreur.

```
    # ----- variables calculées -----
    v["published_at"] = pd.to_datetime(v["published_at"])
    ref = v["collected_at"].fillna(pd.Timestamp(utcnow()))
    v["age_heures"] = ((ref - v["published_at"]).dt.total_seconds() / 3600).clip(lower=0.1)
    v["categorie"] = v["category_id"].map(CATEGORIES).fillna("Autre")
    v["heure_publication"] = v["published_at"].dt.hour
    v["jour_publication"] = v["published_at"].dt.day_name()
    v["duree_min"] = v["duration_seconds"] / 60
    v["nb_tags"] = v["tags"].fillna("").apply(lambda t: len([x for x in t.split("|") if x]))
    v["longueur_titre"] = v["title"].fillna("").str.len()
    vues = v["view_count"].replace(0, np.nan)
    v["taux_engagement"] = (v["like_count"].fillna(0) + v["comment_count"].fillna(0)) / vues
    v["vues_par_heure"] = v["view_count"] / v["age_heures"]
    v["ratio_vues_abonnes"] = v["view_count"] / v["subscriber_count"].replace(0, np.nan)
    return v
```

Bloc 5/5 — La vitesse en temps réel

`groupby("video_id")[...].diff()` calcule, pour chaque vidéo, la différence avec le relevé précédent : le temps écoulé et les vues gagnées. Leur rapport donne les **vues par minute**. `.where(...)` ignore les relevés trop rapprochés.

```
def croissance() -> pd.DataFrame:
    """Vitesse de croissance entre deux relevés successifs (vues par minute)."""
    s = releves()
    if s.empty:
        return s
    s["minutes"] = s.groupby("video_id")["collected_at"].diff().dt.total_seconds() / 60
    s["nouvelles_vues"] = s.groupby("video_id")["view_count"].diff()
    # deux relevés trop rapprochés (< 6 s) ne permettent pas de mesurer une vitesse
    s["vues_par_minute"] = (s["nouvelles_vues"] / s["minutes"]).where(s["minutes"] >= 0.1)
    return s


def table_commentaires() -> pd.DataFrame:
    c = lire("comments")
    if c.empty:
        return c
    c["published_at"] = pd.to_datetime(c["published_at"])
    return c
```

Fichier `04_exploration.py` nouveau fichier

L'analyse exploratoire : statistiques, comparaisons par catégorie et par heure, corrélations et classement des vidéos les plus rapides. Les graphiques sont enregistrés dans `figures/`.

Bloc 1/6 — Préparation

`matplotlib.use("Agg")` permet de créer des images sans ouvrir de fenêtre. `os.makedirs(..., exist_ok=True)` crée le dossier s'il n'existe pas.

```
"""
ÉTAPE 4 — Analyse exploratoire (EDA).

Questions :
  - Quelles catégories font le plus de vues ?
  - À quelle heure publier ?
  - Les likes suivent-ils les vues ? (corrélations)
  - Quelle vidéo grandit le plus vite en ce moment ?

Lancement :  python 04_exploration.py   (les graphiques vont dans figures/)
"""

import os
```

```
import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import pandas as pd

from donnees import croissance, releves, table_commentaires, table_videos

os.makedirs("figures", exist_ok=True)
pd.set_option("display.width", 140)
v = table_videos()
if v.empty:
    raise SystemExit("Base vide : lance d'abord les étapes 2 et 3.")
```

Bloc 2/6 — 1. Aperçu et valeurs manquantes

`.isna().mean() * 100` donne le pourcentage de valeurs manquantes par colonne (`True` compte pour 1). `.describe()` calcule d'un coup effectif, moyenne, écart-type, minimum, quartiles et maximum.

```
print("=" * 70, "\n1. APERÇU DES DONNÉES\n" + "=" * 70)
nb_com = len(table_commentaires())
print(f"{len(v)} vidéos | {len(releves())} relevés | {nb_com} commentaires")
print("\nValeurs manquantes (%) :")
print(
    (
        v[
            [
                "view_count",
                "like_count",
                "comment_count",
                "subscriber_count",
                "duration_seconds",
            ]
```

```
        ]
        .isna()
        .mean()
        * 100
    )
    .round(1)
    .to_string()
)

print("\nStatistiques descriptives :")
print(
    v[
        [
            "view_count",
            "like_count",
            "comment_count",
            "duree_min",
            "taux_engagement",
            "subscriber_count",
        ]
    ]
    .describe()
    .round(3)
    .to_string()
)
```

Bloc 3/6 — 2. Comparaison par catégorie

`groupby("categorie").agg(...)` regroupe les vidéos par catégorie et calcule plusieurs indicateurs à la fois, comme un `GROUP BY` SQL. `.plot.barh` dessine un diagramme en barres horizontales.

```
print("\n" + "=" * 70, "\n2. PAR CATÉGORIE\n" + "=" * 70)
cat = (
    v.groupby("categorie")
    .agg(
        nb_videos=("video_id", "count"),
        vues_moyennes=("view_count", "mean"),
        engagement_moyen=("taux_engagement", "mean"),
    )
    .sort_values("vues_moyennes", ascending=False)
)
print(cat.round(4).to_string())
cat["vues_moyennes"].plot.barh(
    figsize=(8, 5), color="#3b6ea8", title="Vues moyennes par catégorie"
)
plt.xlabel("Vues")
plt.tight_layout()
plt.savefig("figures/vues_par_categorie.png", dpi=120)
plt.close()
```

Bloc 4/6 — 3. Heure de publication

On calcule la **médiane** des vues pour chaque heure de publication : la médiane résiste mieux que la moyenne aux vidéos à plusieurs millions de vues.

```
print("\n" + "=" * 70, "\n3. HEURE DE PUBLICATION (UTC)\n" + "=" * 70)
h = v.groupby("heure_publication")["view_count"].median()
print(h.round(0).to_string())
```

Bloc 5/6 — 4. Matrice de corrélation

`corr(method="spearman")` calcule la corrélation de rang entre toutes les paires de variables. `imshow` dessine la matrice en couleurs (rouge = corrélation positive, bleu = négative) et la double boucle écrit la valeur dans chaque case.

```
print("\n" + "=" * 70, "\n4. CORRÉLATIONS\n" + "=" * 70)
cols = [
    "view_count",
    "like_count",
    "comment_count",
    "subscriber_count",
    "duree_min",
    "nb_tags",
    "age_heures",
]
corr = v[cols].corr(method="spearman").round(2)
print(corr.to_string())
fig, ax = plt.subplots(figsize=(7, 6))
im = ax.imshow(corr, cmap="RdBu_r", vmin=-1, vmax=1)
```

```
ax.set_xticks(range(len(cols)), cols, rotation=45, ha="right")
ax.set_yticks(range(len(cols)), cols)
for i in range(len(cols)):
    for j in range(len(cols)):
        ax.text(j, i, corr.iloc[i, j], ha="center", va="center", fontsize=8)
fig.colorbar(im)
ax.set_title("Corrélations (Spearman)")
plt.tight_layout()
plt.savefig("figures/correlations.png", dpi=120)
plt.close()
```

Bloc 6/6 — 5. Qui grandit le plus vite ?

On garde le dernier calcul de vitesse de chaque vidéo et on trie. Puis on trace une courbe par vidéo suivie (`for vid, d in s.groupby(...)` parcourt les groupes), en échelle logarithmique pour voir petites et grandes vidéos sur le même graphique.

```
print("\n" + "=" * 70, "\n5. TEMPS RÉEL : QUI GRANDIT LE PLUS VITE ?\n" + "=" * 70)
g = croissance().dropna(subset=["vues_par_minute"])
if g.empty:
    print("Pas encore assez de relevés : laisse tourner le collecteur quelques minutes.")
else:
    derniere = g.groupby("video_id").tail(1).merge(v[["video_id", "title"]], on="video_id")
    print(
        derniere.sort_values("vues_par_minute", ascending=False)[
            ["title", "view_count", "vues_par_minute"]
        ]
        .head(10)
        .round(1)
        .to_string(index=False)
    )
```

```
    s = releves().merge(v[["video_id", "tracked"]], on="video_id")
    s = s[s["tracked"] == 1]
    fig, ax = plt.subplots(figsize=(10, 5))
    for vid, d in s.groupby("video_id"):
        ax.plot(d["collected_at"], d["view_count"], marker=".", label=vid)
    ax.set_yscale("log")
    ax.set_title("Évolution des vues (vidéos suivies, échelle log)")
    ax.set_ylabel("Vues")
    ax.legend(fontsize=6, ncol=2)
    plt.tight_layout()
    plt.savefig("figures/evolution_vues.png", dpi=120)
    plt.close()

print("\nGraphiques enregistrés dans le dossier figures/")
```

Lancer

```
python 04_exploration.py
```

Résultat attendu (mode DEMO)

```
======================================================================
1. APERÇU DES DONNÉES
======================================================================
40 vidéos | 250 relevés | 1800 commentaires

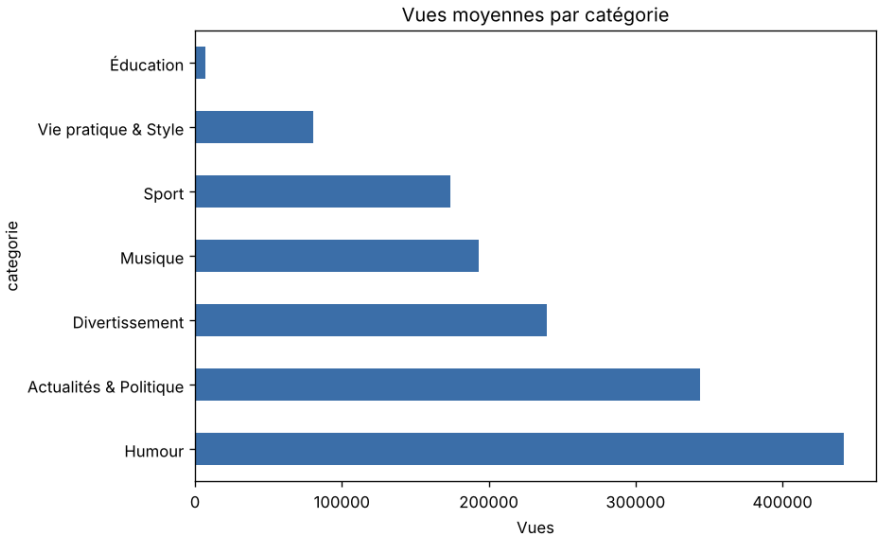
Valeurs manquantes (%) :
view_count          0.0
like_count          0.0
comment_count       0.0
subscriber_count    0.0
duration_seconds    0.0

Statistiques descriptives :
        view_count  like_count  comment_count  duree_min  taux_engagement  subscriber_count
count       40.000      40.000         40.000     40.000           40.000            40.000
mean    257137.550   12465.850       2133.025      7.724            0.056        454058.375
std     471513.016   25628.565       4252.636      6.998            0.018        202797.626
min        255.000       7.000          1.000      0.833            0.026         95407.000
25%       6697.000     342.250         43.500      2.288            0.041        300161.750
50%      41346.500    1495.500        185.500      5.317            0.051        442682.000
75%     189122.250    9233.250       1597.750     10.196            0.066        585900.000
max    1791513.000  111716.000      17595.000     25.933            0.097        800387.000

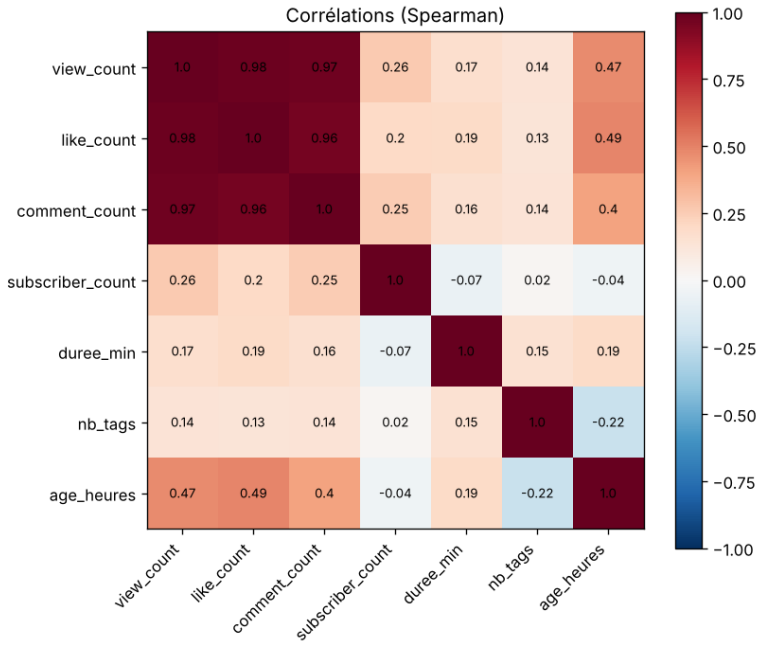
======================================================================
2. PAR CATÉGORIE
======================================================================
                        nb_videos  vues_moyennes  engagement_moyen
categorie
Humour                          8    441529.5000            0.0620
Actualités & Politique          9    343920.6667            0.0469
Divertissement                  3    239097.6667            0.0610
Musique                         9    193097.8889            0.0559
Sport                           5    173959.8000            0.0621
Vie pratique & Style            4     79908.7500            0.0477
Éducation                       2      6686.0000            0.0647

======================================================================
3. HEURE DE PUBLICATION (UTC)
======================================================================
heure_publication
2      187747.0
4      546899.0
6        8754.0
7       39432.0
8       41346.0
10    1082277.0
14    1325262.0

…
```



figures/vues_par_categorie.png (mode DEMO).



figures/correlations.png : likes, vues et commentaires sont presque parfaitement liés.

Questions

1. Pourquoi utiliser la corrélation de Spearman plutôt que celle de Pearson ?
2. Quelle catégorie a le meilleur engagement ? Est-ce celle qui a le plus de vues ?
3. Relancez le script 10 minutes plus tard : le classement des vidéos rapides a-t-il changé ?

5 Classification du sentiment

`python 05_sentiment.py` — 40 min

Objectif : Entraîner un modèle qui reconnaît le sentiment d'un commentaire.

Fichiers à créer : `data/corpus_sentiment.csv`, `05_sentiment.py`

Fichier `data/corpus_sentiment.csv` — nouveau fichier

Le corpus d'apprentissage : 210 commentaires annotés à la main (70 positifs, 70 négatifs, 70 neutres). Format CSV : une colonne `texte` et une colonne `sentiment`. Le fichier complet est dans l'archive du projet ; voici ses premières lignes pour comprendre le format.

Début du fichier

Les textes sont entre guillemets car ils peuvent contenir des virgules. Vous pouvez ajouter vos propres lignes à la fin, avec une des trois étiquettes : `positif`, `negatif`, `neutre` (sans accent).

```
texte,sentiment
"J'adore cette chanson, elle est magnifique",positif
"Bravo à toute l'équipe, travail excellent",positif
"Merci beaucoup pour cette vidéo",positif
"Merci mingi mon frère",positif
"Ezali malamu mingi",positif
"Trop fort, continue comme ça",positif
"Le clip est très beau, félicitations",positif
"Je suis fier de mon pays",positif
"Quelle belle voix, ça me donne des frissons",positif
"Excellente vidéo, très instructive",positif
"Tu m'as trop fait rire hahaha",positif
"Meilleur artiste du Congo",positif
"C'est du lourd !",positif
"Super contenu, je m'abonne",positif
"Les conseils sont vraiment utiles, merci",positif
```

... suite du fichier dans l'archive du projet (data/corpus_sentiment.csv).

Fichier `05_sentiment.py` — nouveau fichier

Entraîne, évalue et enregistre le modèle de sentiment, puis l'applique à tous les commentaires de la base.

Bloc 1/8 — Imports

On importe les outils de scikit-learn : `TfidfVectorizer` (texte vers nombres), `LogisticRegression` (le classifieur), les outils de validation croisée et de mesure, et `Pipeline` qui enchaîne les étapes.

```
"""
ÉTAPE 5 — Classification du sentiment des commentaires (TF-IDF + régression logistique).

  1. entraîne le modèle sur data/corpus_sentiment.csv (commentaires annotés à la main)
  2. l'évalue (validation croisée + matrice de confusion)
  3. prédit le sentiment de tous les commentaires de la base
  4. exporte 100 commentaires réels dans data/a_annoter.csv (exercice d'annotation)

Une fois le modèle enregistré, le collecteur (étape 3) prédit le sentiment
des nouveaux commentaires en temps réel.

Lancement :  python 05_sentiment.py
"""
```

```
import os

import joblib
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.model_selection import StratifiedKFold, cross_val_predict
from sklearn.pipeline import FeatureUnion, Pipeline
from sqlalchemy import bindparam, update

from db import comments, get_engine
from donnees import table_commentaires, table_videos
from pretraitement import MODEL_PATH, nettoyer, predire
```

Bloc 2/8 — 1. Charger le corpus

On lit le CSV avec `pd.read_csv`. Si les étudiants ont annoté `a_annoter.csv`, ces lignes sont ajoutées (`pd.concat`). X contient les textes nettoyés (les entrées), y les étiquettes (les réponses attendues).

```
# ---------- 1. corpus d'entraînement ----------
corpus = pd.read_csv("data/corpus_sentiment.csv")
# Si les étudiants ont annoté des commentaires réels, on les ajoute
if os.path.exists("data/a_annoter.csv"):
    ann = pd.read_csv("data/a_annoter.csv").dropna(subset=["sentiment"])
    ann = ann[ann["sentiment"].isin(["positif", "negatif", "neutre"])]
    if len(ann):
        print(f"+ {len(ann)} commentaires annotés par les étudiants ajoutés au corpus")
        corpus = pd.concat([corpus, ann[["texte", "sentiment"]]], ignore_index=True)

X = corpus["texte"].map(nettoyer)
y = corpus["sentiment"]
print(f"Corpus : {len(corpus)} phrases\n{y.value_counts().to_string()}\n")
```

Bloc 3/8 — Le modèle : un Pipeline

Un **Pipeline** enchaîne la vectorisation et la classification : on peut ensuite appeler `modele.fit()` et `modele.predict()` directement sur du texte. `FeatureUnion` combine deux vectoriseurs : les mots (`ngram_range=(1, 2)` = mots seuls et paires de mots) et les morceaux de 2 à 5 lettres (`analyzer="char_wb"`). `class_weight="balanced"` donne le même poids à chaque classe.

```
# Deux familles de variables : les mots (et paires de mots) + des morceaux de
# 2 à 5 lettres, plus robustes aux fautes d'orthographe et au lingala.
modele = Pipeline(
    [
        (
            "tfidf",
            FeatureUnion(
                [
                    ("mots", TfidfVectorizer(ngram_range=(1, 2), sublinear_tf=True)),
                    (
                        "lettres",
                        TfidfVectorizer(
                            analyzer="char_wb", ngram_range=(2, 5), sublinear_tf=True
                        ),
                    ),
                ]
            ),
        ),
        ("clf", LogisticRegression(max_iter=3000, class_weight="balanced", C=2)),
    ]
)
```

Bloc 4/8 — 2. Évaluer avec la validation croisée

`cross_val_predict` prédit chaque phrase avec un modèle entraîné sans elle (5 plis). `classification_report` affiche précision, rappel et F1. On affiche aussi la matrice de confusion et quelques erreurs, très utiles pour la discussion.

```
# ---------- 2. évaluation (validation croisée à 5 plis) ----------
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
pred = cross_val_predict(modele, X, y, cv=cv)
print("Évaluation (validation croisée) :")
print(classification_report(y, pred, zero_division=0))
labels = ["negatif", "neutre", "positif"]
print("Matrice de confusion (lignes = vrai, colonnes = prédit) :")
print(
    pd.DataFrame(
        confusion_matrix(y, pred, labels=labels), index=labels, columns=labels
    ).to_string()
)

erreurs = corpus[pred != y].assign(predit=pred[pred != y])
if len(erreurs):
    print("\nExemples d'erreurs :")
    print(erreurs.head(8).to_string(index=False))
```

Bloc 5/8 — 3. Entraîner et enregistrer

Une fois évalué, le modèle est entraîné sur **tout** le corpus puis enregistré avec `joblib.dump`. On affiche les mots qui pèsent le plus pour chaque classe : `coef_` contient les poids appris, `argsort()` les trie.

```
# ---------- 3. entraînement final + prédiction sur la base ----------
modele.fit(X, y)
os.makedirs("models", exist_ok=True)
joblib.dump(modele, MODEL_PATH)
print(f"\nModèle enregistré : {MODEL_PATH}")

coefs = modele.named_steps["clf"].coef_
noms = modele.named_steps["tfidf"].get_feature_names_out()
est_mot = [n.startswith("mots__") for n in noms]
for i, classe in enumerate(modele.named_steps["clf"].classes_):
    top = [j for j in coefs[i].argsort()[::-1] if est_mot[j]][:8]
    print(f"Mots les plus liés à « {classe} » : {', '.join(noms[j][6:] for j in top)}")
```

Bloc 6/8 — Prédire les commentaires de la base

On prédit tous les commentaires collectés, puis on met à jour la base en une seule commande : `bindparam` crée des « trous » dans la requête, remplis par la liste de dictionnaires (une mise à jour par commentaire).

```
c = table_commentaires()
if c.empty:
    print("\nAucun commentaire en base : lance le collecteur (étape 3).")
    raise SystemExit
labs, scores = predire(modele, c["text"].tolist())
c["sentiment"], c["sentiment_score"] = labs, scores
eng = get_engine()
with eng.begin() as conn:
    conn.execute(
        update(comments)
        .where(comments.c.comment_id == bindparam("cid"))
        .values(sentiment=bindparam("s"), sentiment_score=bindparam("sc")),
        [{"cid": a, "s": b, "sc": d} for a, b, d in zip(c["comment_id"], labs, scores)],
    )
```

```
print(f"\n{len(c)} commentaires classés :")
print(
    c["sentiment"]
    .value_counts(normalize=True)
    .mul(100)
    .round(1)
    .astype(str)
    .add(" %")
    .to_string()
)
```

Bloc 7/8 — Le sentiment moyen par vidéo

On code positif = +1, neutre = 0, négatif = −1, puis on fait la moyenne par vidéo. On mesure enfin la corrélation entre ce sentiment moyen et le taux d'engagement.

```
# Sentiment moyen par vidéo (score : positif = +1, neutre = 0, négatif = -1)
c["score_num"] = c["sentiment"].map({"positif": 1, "neutre": 0, "negatif": -1})
v = table_videos()[["video_id", "title", "taux_engagement"]]
par_video = (
    c.groupby("video_id")
    .agg(nb=("comment_id", "count"), sentiment_moyen=("score_num", "mean"))
    .reset_index()
    .merge(v, on="video_id")
)
print("\nSentiment moyen par vidéo :")
print(
    par_video.sort_values("sentiment_moyen", ascending=False)[
        ["title", "nb", "sentiment_moyen", "taux_engagement"]
    ]
```

```
    .round(3)
    .to_string(index=False)
)
print(
    "\nCorrélation sentiment moyen / engagement :",
    round(
        par_video["sentiment_moyen"].corr(par_video["taux_engagement"], method="spearman"), 3
    ),
)
```

Bloc 8/8 — 4. Le fichier à annoter

`c.sample(100)` tire 100 commentaires au hasard (`random_state=1` rend le tirage reproductible). La colonne `sentiment` est laissée vide pour les étudiants.

```
# ---------- 4. fichier d'annotation pour les étudiants ----------
if not os.path.exists("data/a_annoter.csv"):
    ech = c.sample(min(100, len(c)), random_state=1)[["text", "sentiment"]]
    ech = ech.rename(columns={"text": "texte", "sentiment": "prediction_modele"})
    ech["sentiment"] = ""
    ech.to_csv("data/a_annoter.csv", index=False)
    print(
        "\nFichier data/a_annoter.csv créé : remplissez la colonne 'sentiment' "
        "(positif / negatif / neutre) puis relancez ce script."
    )
```

Lancer

```
python 05_sentiment.py
```

Résultat attendu (mode DEMO)

```
Corpus : 210 phrases
sentiment
positif    70
negatif    70
neutre     70

Évaluation (validation croisée) :
              precision    recall  f1-score   support

     negatif       0.71      0.64      0.68        70
      neutre       0.77      0.94      0.85        70
     positif       0.80      0.70      0.75        70

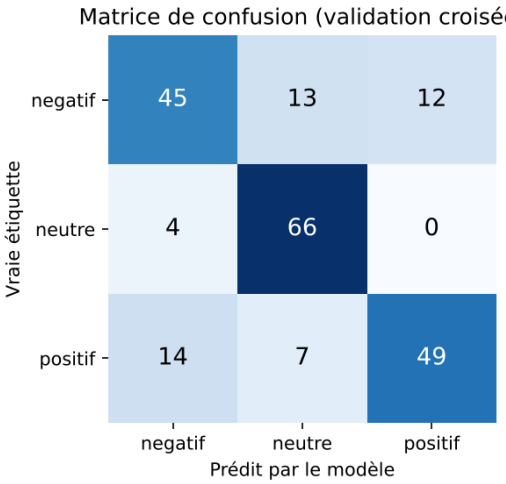
    accuracy                           0.76       210
   macro avg       0.76      0.76      0.76       210
weighted avg       0.76      0.76      0.76       210

Matrice de confusion (lignes = vrai, colonnes = prédit) :
          negatif  neutre  positif
negatif       45      13       12
neutre         4      66        0
positif       14       7       49

Exemples d'erreurs :
                                texte sentiment  predit
                     Ezali malamu mingi   positif negatif
            Trop fort, continue comme ça   positif negatif
                Je suis fier de mon pays   positif  neutre
                       C'est du lourd !   positif  neutre
                   La danse est géniale   positif  neutre
    Je regarde en boucle, c'est trop bien   positif  neutre
Félicitations pour les un million de vues   positif negatif
                  Magnifique, rien à dire   positif negatif

Modèle enregistré : /tmp/claude-0/-home-claude/874eac73-ba9f-5908-8b2e-aab60927b09b/scratchpad/run/models/sent…
Mots les plus liés à « negatif » : pas, decu, mauvais, inutile, ne, nul, lamentable, rien
Mots les plus liés à « neutre » : la, present, mbote, le, je, ok, combien, qui
Mots les plus liés à « positif » : merci, bravo, adore, bonne, tres, enorme, fan, suis fan

1800 commentaires classés :
sentiment
positif    43.3 %
…
```



Matrice de confusion : environ 76 % de bonnes réponses.

Activer le sentiment en temps réel

Arrêtez le collecteur (Ctrl + C) et relancez-le : il affiche maintenant « Sentiment en temps réel : activé ».

Exercice d'annotation

Ouvrez `data/a_annoter.csv` (Excel ou VS Code), remplissez la colonne `sentiment` pour 50 commentaires, enregistrez, puis relancez `python 05_sentiment.py`. Comparez les scores.

Questions

1. Quelle classe est la mieux reconnue ? Laquelle est la plus confondue, et avec quoi ?
2. Pourquoi « C'est du lourd ! » est-il difficile pour le modèle ?
3. Le sentiment moyen d'une vidéo est-il lié à son engagement ?

6 Clustering des vidéos

`python 06_clustering.py --k 3` — 30 min

Objectif : Regrouper les vidéos selon leur profil de croissance.

Fichiers à créer : `06_clustering.py`

Fichier `06_clustering.py` — nouveau fichier

Regroupe les vidéos suivies selon leur profil de croissance avec K-Means, propose une interprétation de chaque groupe et dessine les groupes en 2D.

Bloc 1/9 — Imports et options

KMeans, PCA (ACP), `silhouette_score` et `StandardScaler` viennent de scikit-learn. L'option `--k` fixe le nombre de groupes (3 par défaut).

```
"""
ÉTAPE 6 — Clustering des vidéos selon leur profil de croissance (K-Means).

Variables (calculées à partir des relevés en temps réel) :
  - vitesse    : vues par minute sur la dernière période
  - accel      : la vitesse augmente-t-elle ou diminue-t-elle ?
  - engagement, likes/vues, commentaires/vues
  - age        : âge de la vidéo (log)

Lancement :  python 06_clustering.py --k 3
"""

import argparse
import os
```

```
import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler
from sqlalchemy import update

from db import get_engine, videos
from donnees import croissance, table_videos

parser = argparse.ArgumentParser()
parser.add_argument(
    "--k",
    type=int,
    default=3,
    help="nombre de groupes (0 = choix automatique par la silhouette)",
)
args = parser.parse_args()
os.makedirs("figures", exist_ok=True)
```

Bloc 2/9 — Vérifier qu'il y a assez de relevés

Il faut plusieurs vitesses par vidéo pour décrire sa croissance. Sinon, on arrête avec un message clair.

```
g = croissance().dropna(subset=["vues_par_minute"])
v = table_videos()
v = v[v["tracked"] == 1]
if g.empty or g.groupby("video_id").size().max() < 3:
    raise SystemExit(
        "Il faut au moins 4 relevés par vidéo : laisse tourner le collecteur un peu plus."
    )
```

Bloc 3/9 — Décrire la croissance d'une vidéo

Pour une vidéo, on coupe ses relevés en deux moitiés et on compare la vitesse moyenne du début et de la fin : c'est l'**accélération**. `clip(lower=0)` remplace les vitesses négatives par 0.

```
def profil(d: pd.DataFrame) -> pd.Series:
    d = d.sort_values("collected_at")
    # YouTube retire parfois des vues (fraude) : une vitesse négative est ramenée à 0
    d = d.assign(vues_par_minute=d["vues_par_minute"].clip(lower=0))
    moitie = max(1, len(d) // 2)
    debut, fin = (
        d["vues_par_minute"].iloc[:moitie].mean(),
        d["vues_par_minute"].iloc[moitie:].mean(),
    )
    return pd.Series({"vitesse": fin, "accel": (fin - debut) / (debut + 1)})
```

Bloc 4/9 — Construire le tableau des variables

`groupby(...).apply(profil)` applique la fonction à chaque vidéo. On ajoute l'engagement et l'âge, puis on passe la vitesse et l'âge au logarithme (`np.log1p` = log(1 + x), qui accepte 0).

```
feat = g.groupby("video_id").apply(profil, include_groups=False).reset_index()
feat = feat.merge(
    v[
        [
            "video_id",
            "title",
            "taux_engagement",
            "like_count",
            "comment_count",
            "view_count",
            "age_heures",
        ]
    ],
    on="video_id",
```

```
)
feat["likes_par_vue"] = feat["like_count"] / feat["view_count"]
feat["coms_par_vue"] = feat["comment_count"] / feat["view_count"]
feat["log_vitesse"] = np.log1p(feat["vitesse"].clip(lower=0))
feat["log_age"] = np.log1p(feat["age_heures"])
cols = ["log_vitesse", "accel", "taux_engagement", "likes_par_vue", "coms_par_vue", "log_age"]
feat = feat.dropna(subset=cols)
if len(feat) < 4:
    raise SystemExit("Pas assez de vidéos exploitables pour le clustering (minimum 4).")
```

Bloc 5/9 — Standardiser

`fit_transform` ramène chaque variable à une moyenne de 0 et un écart-type de 1, pour qu'aucune ne domine les autres.

```
Xs = StandardScaler().fit_transform(feat[cols])
```

Bloc 6/9 — Choisir le nombre de groupes

Pour k = 2 à 6, on lance K-Means et on affiche l'inertie et la silhouette. `n_init=10` relance l'algorithme 10 fois avec des départs différents et garde le meilleur résultat. `args.k or max(...)` : si `--k 0`, on prend le k de meilleure silhouette.

```
# ---------- choix de k : méthode du coude + silhouette ----------
print("k | inertie | silhouette")
res = {}
for k in range(2, min(7, len(feat) // 2 + 1)):
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(Xs)
    res[k] = silhouette_score(Xs, km.labels_)
    print(f"{k} | {km.inertia_:8.1f} | {res[k]:.3f}")
k = args.k or max(res, key=res.get)
print(
    f"\n→ k retenu : {k}  (avec peu de vidéos, la silhouette favorise les grands k : "
    "préférer un k interprétable)"
)
```

Bloc 7/9 — Le clustering final

On relance K-Means avec le k choisi. `km.labels_` donne le numéro de groupe de chaque vidéo.

```
km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(Xs)
feat["cluster"] = km.labels_
```

■ Bloc 8/9 — Interpréter les groupes

On calcule le profil moyen de chaque groupe, puis `nommer()` lui donne un nom selon des règles simples (vitesse quasi nulle, accélération, vitesse maximale). Les groupes sont ensuite enregistrés dans la colonne `cluster` de la table videos.

```
# ---------- interprétation ----------
resume = feat.groupby("cluster")[["vitesse", "accel", "taux_engagement", "age_heures"]].mean()
resume["nb_videos"] = feat.groupby("cluster").size()


def nommer(r):
    vmax = resume["vitesse"].max()
    if r["vitesse"] < 0.02 * vmax:
        return "stable / saturée"
    if r["accel"] > 0.2:
        return "en accélération"
    if r["vitesse"] == vmax:
        return "explosive"
    return "croissance modérée"
```

```
resume["interpretation"] = resume.apply(nommer, axis=1)
print("\nProfil moyen de chaque groupe :")
print(resume.round(3).to_string())
for c, d in feat.groupby("cluster"):
    print(f"\nGroupe {c} ({resume.loc[c, 'interpretation']}) :")
    for t in d["title"].head(6):
        print("   -", t[:70])

with get_engine().begin() as conn:
    for vid, c in zip(feat["video_id"], feat["cluster"]):
        conn.execute(update(videos).where(videos.c.video_id == vid).values(cluster=int(c)))
```

■ Bloc 9/9 — Dessiner les groupes

L'ACP résume les 6 variables en 2 axes ; on dessine un nuage de points coloré par groupe.

```
# ---------- visualisation en 2D (ACP) ----------
xy = PCA(n_components=2).fit_transform(Xs)
fig, ax = plt.subplots(figsize=(8, 6))
for c in sorted(feat["cluster"].unique()):
    m = feat["cluster"] == c
    ax.scatter(xy[m, 0], xy[m, 1], s=60, label=f"{c} : {resume.loc[c, 'interpretation']}")
ax.set_title("Clusters de vidéos (projection ACP)")
ax.legend()
plt.tight_layout()
plt.savefig("figures/clusters.png", dpi=120)
print("\nGraphique : figures/clusters.png — clusters enregistrés dans la table videos")
```

Lancer

```
python 06_clustering.py --k 3
```

Résultat attendu (mode DEMO)

```
k | inertie | silhouette
2 |     61.2 | 0.349
3 |     36.6 | 0.350
4 |     20.6 | 0.440
5 |     13.7 | 0.472
6 |      7.2 | 0.494

→ k retenu : 3  (avec peu de vidéos, la silhouette favorise les grands k : préférer un k interprétable)

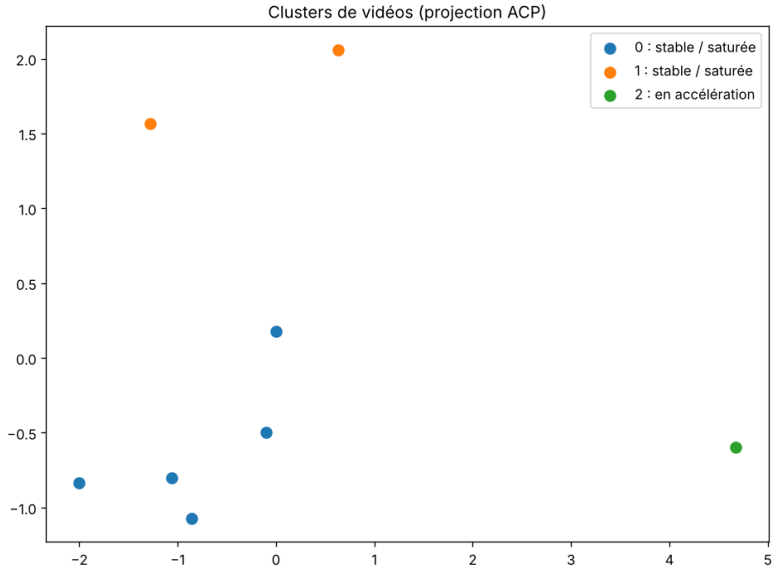
Profil moyen de chaque groupe :
           vitesse      accel  taux_engagement  age_heures  nb_videos      interpretation
cluster
0            0.000     -0.089            0.049     123.018          7    stable / saturée
1         3162.653     -0.007            0.053       0.657          7  croissance modérée
2        85714.286  85714.286            0.083      96.256          1     en accélération

Groupe 0 (stable / saturée) :
   - Micro-trottoir au marché Total #5
   - Comment réussir son bac en 3 mois #21
   - Prank au quartier Bacongo #25
   - Interview exclusive d'un artiste #29
   - Les meilleurs buts de la semaine #30
   - Débat : la jeunesse et le chômage #33

Groupe 1 (croissance modérée) :
   - Mariage coutumier à Brazzaville #0
   - Concert live au Palais des congrès #2
   - Les meilleurs buts de la semaine #3
   - Ndombolo 2026 : la danse qui enflamme Brazzaville #6
   - Vlog : une journée à Oyo #9
   - Diables Rouges : résumé du match #32

Groupe 2 (en accélération) :
   - Débat : la jeunesse et le chômage #7

Graphique : figures/clusters.png — clusters enregistrés dans la table videos
```



figures/clusters.png (mode DEMO).

Questions

1. Pourquoi passe-t-on la vitesse au logarithme ?
2. Que se passerait-il sans standardisation ?
3. Une vidéo forme-t-elle un groupe à elle seule ? Comparez avec l'étape 8.

7 Prédiction des vues

`python 07_prediction.py` — 20 min

Objectif : Prédire les vues et la viralité d'une vidéo.

Fichiers à créer : `07_prediction.py`

Fichier `07_prediction.py` — nouveau fichier

Prédit le nombre de vues (régression) et le caractère « viral » d'une vidéo (classification), en comparant plusieurs modèles.

Bloc 1/4 — Imports

On importe deux modèles de régression, une forêt aléatoire de classification, et les mesures R² et erreur absolue moyenne.

```
"""
ÉTAPE 7 — Prédiction : combien de vues ? virale ou non ?

  A. Régression     : prédire log(vues) — Random Forest vs régression linéaire
  B. Classification : prédire « virale » (vues > 2 × abonnés de la chaîne)

Attention : on n'utilise PAS les likes ni les commentaires comme variables,
car ils sont une conséquence des vues (ce serait de la « fuite de données »).

Lancement :  python 07_prediction.py
"""

import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.metrics import classification_report, mean_absolute_error, r2_score
from sklearn.model_selection import cross_val_predict, KFold, StratifiedKFold

from donnees import table_videos
```

Bloc 2/4 — Préparer les variables

On garde les vidéos complètes, on vérifie qu'il y en a assez, puis on crée les variables en logarithme. `pd.get_dummies` transforme la catégorie (texte) en colonnes 0/1 : une par catégorie. C'est l'**encodage one-hot**, nécessaire car les modèles ne comprennent que des nombres.

```
v = table_videos().dropna(subset=["view_count", "duration_seconds"])
if len(v) < 15:
    raise SystemExit(
        f"Seulement {len(v)} vidéos : il en faut au moins 15 "
        "(étape 2 avec la tendance ou --search)."
    )

v["log_abonnes"] = np.log1p(v["subscriber_count"].fillna(v["subscriber_count"].median()))
v["log_age"] = np.log1p(v["age_heures"])
v["log_duree"] = np.log1p(v["duration_seconds"])
v["est_weekend"] = v["published_at"].dt.dayofweek >= 5
X = pd.concat(
    [
        v[
```

```
            [
                "log_abonnes",
                "log_age",
                "log_duree",
                "nb_tags",
                "longueur_titre",
                "heure_publication",
                "est_weekend",
            ]
        ],
        pd.get_dummies(v["categorie"], prefix="cat"),
    ],
    axis=1,
).astype(float)
print(f"{len(v)} vidéos, {X.shape[1]} variables explicatives\n")
```

■ Bloc 3/4 — A. Régression

La cible est `log10(vues)`. La boucle évalue chaque modèle par validation croisée (`KFold` à 5 plis). `10 ** erreur` traduit l'erreur en facteur multiplicatif (×2 = « se trompe d'un facteur 2 »). On affiche ensuite l'importance des variables de la forêt.

```
# ---------- A. Régression ----------
y = np.log10(v["view_count"].clip(lower=1))
cv = KFold(5, shuffle=True, random_state=0)
print("A. RÉGRESSION — prédire log10(vues)")
for nom, m in [
    ("Régression linéaire", LinearRegression()),
    ("Random Forest", RandomForestRegressor(300, min_samples_leaf=2, random_state=0)),
]:
    p = cross_val_predict(m, X, y, cv=cv)
    print(
        f"  {nom:20} R² = {r2_score(y, p):.3f} "
        f"| erreur moyenne = ×{10 ** mean_absolute_error(y, p):.2f}"
    )
rf = RandomForestRegressor(300, min_samples_leaf=2, random_state=0).fit(X, y)
imp = pd.Series(rf.feature_importances_, index=X.columns).sort_values(ascending=False)
print("\n  Variables les plus importantes :")
print(imp.head(8).round(3).to_string())
```

■ Bloc 4/4 — B. Classification

Une vidéo est « virale » si ses vues dépassent 2 fois les abonnés de sa chaîne. `.astype(int)` convertit vrai/faux en 1/0. `StratifiedKFold` garde la même proportion de virales dans chaque pli.

```
# ---------- B. Classification ----------
v["virale"] = (v["view_count"] > 2 * v["subscriber_count"].fillna(np.inf)).astype(int)
print(
    f"\nB. CLASSIFICATION — vidéo « virale » (vues > 2 × abonnés) : "
    f"{v['virale'].sum()} virales sur {len(v)}"
)
if v["virale"].nunique() < 2 or v["virale"].value_counts().min() < 3:
    print(
        "  Pas assez d'exemples dans chaque classe ; "
        "change la définition de « virale » (ex. > abonnés)."
    )
else:
    clf = RandomForestClassifier(
        300, min_samples_leaf=2, class_weight="balanced", random_state=0
```

```
    )
    n = min(5, v["virale"].value_counts().min())
    p = cross_val_predict(
        clf, X, v["virale"], cv=StratifiedKFold(n, shuffle=True, random_state=0)
    )
    print(
        classification_report(
            v["virale"], p, target_names=["non virale", "virale"], zero_division=0
        )
    )

print(
    "Questions : quelle variable compte le plus ? "
    "Le modèle est-il meilleur qu'une simple moyenne (R² > 0) ?"
)
```

Lancer

```
python 07_prediction.py
```

Résultat attendu (mode DEMO)

```
40 vidéos, 14 variables explicatives

A. RÉGRESSION — prédire log10(vues)
  Régression linéaire  R² = 0.047 | erreur moyenne = ×5.77
  Random Forest        R² = 0.321 | erreur moyenne = ×4.40

  Variables les plus importantes :
log_age                       0.634
log_abonnes                   0.100
longueur_titre                0.073
log_duree                     0.071
nb_tags                       0.066
est_weekend                   0.029
heure_publication             0.009
cat_Actualités & Politique    0.008

B. CLASSIFICATION — vidéo « virale » (vues > 2 × abonnés) : 4 virales sur 40
              precision    recall  f1-score   support

  non virale       0.94      0.92      0.93        36
      virale       0.40      0.50      0.44         4

    accuracy                           0.88        40
   macro avg       0.67      0.71      0.69        40
weighted avg       0.89      0.88      0.88        40

Questions : quelle variable compte le plus ? Le modèle est-il meilleur qu'une simple moyenne (R² > 0) ?
```

Attention

On n'utilise **ni les likes ni les commentaires** comme variables : ils grandissent en même temps que les vues. Le modèle semblerait excellent mais serait inutile pour une vidéo qui vient de sortir (**fuite de données**, exercice F).

Questions

1. Quelle variable est la plus importante ? Est-ce surprenant ?
2. Que signifie un R² négatif ?
3. Avec 50 vidéos, peut-on faire confiance à ces résultats ?

8 Détection d'anomalies

`python 08_anomalies.py` — 15 min

Objectif : Repérer les pics de vues et les comptes suspects.

Fichiers à créer : `08_anomalies.py`

Fichier `08_anomalies.py` — nouveau fichier

Détecte les hausses de vues anormales (z-score robuste) et les commentateurs suspects (Isolation Forest).

Bloc 1/6 — Imports

`IsolationForest` est l'algorithme de détection d'anomalies de scikit-learn.

```
"""
ÉTAPE 8 — Détection d'anomalies.

  A. Hausses de vues anormales (z-score robuste sur la vitesse de croissance)
     -> vues achetées ? partage massif soudain ?
  B. Commentateurs suspects (Isolation Forest sur des profils ANONYMISÉS)
     -> comptes récents, sans vidéos, qui postent beaucoup, avec des liens...

Lancement :  python 08_anomalies.py
"""

import numpy as np
import pandas as pd
from sklearn.ensemble import IsolationForest
from sqlalchemy import bindparam, update

from db import comments, get_engine, utcnow
from donnees import croissance, lire, table_commentaires, table_videos
```

Bloc 2/6 — A. Les hausses de vues anormales

La fonction `zrobuste` est définie à l'intérieur du `else` : elle compare chaque vitesse à la médiane de la vidéo, divisée par l'écart absolu médian (avec un plancher). `transform` l'applique vidéo par vidéo. Le filtre `(z > 5) & (nouvelles_vues >= 500)` combine deux conditions avec `&` (les parenthèses sont obligatoires).

```
# ---------- A. pics de vues ----------
print("A. HAUSSES DE VUES ANORMALES\n")
g = croissance().dropna(subset=["vues_par_minute"])
if g.empty:
    print("Pas assez de relevés.")
else:

    def zrobuste(x):
        """z-score robuste : (x - médiane) / MAD. Le plancher évite de diviser
        par ~0 pour les vidéos saturées qui ne gagnent presque plus de vues."""
        med = x.median()
        mad = max((x - med).abs().median() * 1.4826, 0.5 * abs(med), 10)
        return (x - med) / mad
```

```
    g["z"] = g.groupby("video_id")["vues_par_minute"].transform(zrobuste)
    # anomalie = vitesse très au-dessus de l'habitude de LA vidéo
    # ET au moins 500 vues d'un coup
    pics = g[(g["z"] > 5) & (g["nouvelles_vues"] >= 500)].merge(
        table_videos()[["video_id", "title"]], on="video_id"
    )
    if pics.empty:
        print("Aucune hausse anormale détectée.")
    else:
        aff = pics.sort_values("z", ascending=False).head(10)
        aff = aff.assign(vues_par_minute=aff["vues_par_minute"].round(0), z=aff["z"].round(1))
        print(
            aff[["collected_at", "title", "nouvelles_vues", "vues_par_minute", "z"]].to_string(
                index=False
            )
        )
```

Bloc 3/6 — B. Les variables de chaque commentaire

`str.contains` repère les liens, la fonction `lambda` calcule la part de majuscules d'un texte, `str.len` sa longueur.

```
# ---------- B. commentateurs suspects ----------
print("\nB. COMMENTATEURS SUSPECTS\n")
c = table_commentaires()
p = lire("commenters")
if c.empty or p.empty:
    raise SystemExit("Pas encore de commentaires / profils : laisse tourner le collecteur.")

c["a_un_lien"] = c["text"].str.contains(r"http|www\.|\.com|\.xyz", case=False, regex=True)
c["majuscules"] = c["text"].apply(lambda t: sum(ch.isupper() for ch in t) / max(len(t), 1))
c["longueur"] = c["text"].str.len()
```

Bloc 4/6 — Le profil de chaque auteur

On agrège par auteur : nombre de commentaires, de vidéos commentées, part de liens, de majuscules, nombre de textes différents. Le **taux de répétition** vaut 0 si tous les messages sont différents, et s'approche de 1 s'il répète le même message. On ajoute son profil public (ancienneté, abonnés, vidéos).

```
auteurs = (
    c.groupby("author_hash")
    .agg(
        nb_commentaires=("comment_id", "count"),
        nb_videos_commentees=("video_id", "nunique"),
        part_liens=("a_un_lien", "mean"),
        part_majuscules=("majuscules", "mean"),
        longueur_moy=("longueur", "mean"),
        textes_differents=("text", "nunique"),
    )
    .reset_index()
)
auteurs["taux_repetition"] = 1 - auteurs["textes_differents"] / auteurs["nb_commentaires"]
auteurs = auteurs.merge(p, on="author_hash", how="left")
auteurs["anciennete_jours"] = (
    pd.Timestamp(utcnow()) - pd.to_datetime(auteurs["channel_created_at"])
).dt.days
auteurs["log_abonnes"] = np.log1p(auteurs["subscriber_count"].fillna(0))
auteurs["log_videos"] = np.log1p(auteurs["video_count"].fillna(0))
auteurs["log_anciennete"] = np.log1p(
    auteurs["anciennete_jours"].fillna(auteurs["anciennete_jours"].median())
)
```

Bloc 5/6 — Isolation Forest

`contamination=0.03` indique qu'on s'attend à environ 3 % d'anomalies. `fit_predict` renvoie −1 pour une anomalie et 1 sinon. On compare ensuite le profil moyen des suspects et des normaux.

```
cols = [
    "nb_commentaires",
    "nb_videos_commentees",
    "part_liens",
    "part_majuscules",
    "taux_repetition",
    "log_abonnes",
    "log_videos",
    "log_ancienneté",
]
iso = IsolationForest(n_estimators=300, contamination=0.03, random_state=0)
auteurs["suspect"] = iso.fit_predict(auteurs[cols].fillna(0)) == -1
auteurs["score_anomalie"] = -iso.score_samples(auteurs[cols].fillna(0))
```

```
print(f"{len(auteurs)} commentateurs analysés, {auteurs['suspect'].sum()} suspects")
print("\nProfil moyen : suspects vs normaux")
print(
    auteurs.groupby("suspect")[
        [
            "nb_commentaires",
            "part_liens",
            "taux_repetition",
            "ancienneté_jours",
            "subscriber_count",
            "video_count",
        ]
    ]
    .mean()
    .round(2)
    .to_string()
)
```

Bloc 6/6 — Exemples et enregistrement

On affiche des commentaires écrits par les suspects, on enregistre l'indicateur `is_suspect` dans la base, puis on montre la répartition des pays déclarés (et la part de « non déclaré »).

```
susp = auteurs[auteurs["suspect"]].sort_values("score_anomalie", ascending=False)
exemples = (
    c[c["author_hash"].isin(susp["author_hash"])].drop_duplicates("text")["text"].head(8)
)
print("\nExemples de commentaires d'auteurs suspects :")
for t in exemples:
    print("  -", t[:90])

ids_suspects = set(susp["author_hash"])
with get_engine().begin() as conn:
    conn.execute(
        update(comments)
        .where(comments.c.comment_id == bindparam("cid"))
        .values(is_suspect=bindparam("s")),
```

```
        [
            {"cid": cid, "s": int(a in ids_suspects)}
            for cid, a in zip(c["comment_id"], c["author_hash"])
        ],
    )

print("\nPays DÉCLARÉS par les commentateurs (beaucoup ne déclarent rien) :")
print(
    p["country"]
    .fillna("non déclaré")
    .value_counts(normalize=True)
    .mul(100)
    .round(1)
    .head(8)
    .to_string()
)
print("\nRésultats enregistrés (colonne comments.is_suspect).")
```

Lancer

```
python 08_anomalies.py
```

Résultat attendu (mode DEMO)

```
A. HAUSSES DE VUES ANORMALES

       collected_at                              title  nouvelles_vues  vues_par_minute       z
2026-10-28 08:41:06 Débat : la jeunesse et le chômage #7         60000.0         600000.0 60000.0

B. COMMENTATEURS SUSPECTS

396 commentateurs analysés, 12 suspects

Profil moyen : suspects vs normaux
         nb_commentaires  part_liens  taux_repetition  anciennete_jours  subscriber_count  video_count
suspect
False               4.59         0.0             0.03           2077.58              6.99         1.10
True               10.67         0.5             0.53            360.42              0.00         5.25

Exemples de commentaires d'auteurs suspects :
  - Cliquez ici pour des vues gratuites http://vues-gratuites.xyz
  - Promo crypto : doublez votre argent maintenant http://crypto-promo.top
  - GAGNEZ 500 000 FCFA PAR JOUR, contactez-moi WhatsApp
  - Quelle voix, c'est incroyable
  - Abonne-toi à moi je m'abonne à toi !!!
  - Abonnez-vous à ma chaîne !!! http://bit.ly/gagner-argent
  - Quelle perte de temps
  - Vidéo trop longue et ennuyeuse

Pays DÉCLARÉS par les commentateurs (beaucoup ne déclarent rien) :
country
non déclaré    51.8
CG             16.4
CD             11.1
CI              8.8
CM              6.1
FR              5.8

Résultats enregistrés (colonne comments.is_suspect).
```

À retenir

Une alerte n'est pas une preuve. Un fan très actif peut être signalé : c'est un **faux positif**. La détection d'anomalies sert à trier ce qu'un humain doit vérifier.

Questions

1. Pourquoi la médiane plutôt que la moyenne pour détecter un pic ?
2. Trouvez un faux positif parmi les commentateurs suspects et expliquez-le.
3. Quelle part des commentateurs ne déclare aucun pays ? Qu'en concluez-vous ?

9 Démographie : âge, genre, pays

`python 09_demographie.py --jours 90` — 15 min

Objectif : Obtenir la répartition de l'audience de sa propre chaîne.

Fichiers à créer : `09_demographie.py`

Fichier `09_demographie.py` — nouveau fichier

Récupère l'âge, le genre et le pays de l'audience d'une chaîne dont on est propriétaire (YouTube Analytics), ou des données simulées en mode DEMO.

Bloc 1/6 — Documentation et imports

La docstring détaille la préparation OAuth. Les bibliothèques Google ne sont importées que dans le mode réel, pour que le mode DEMO fonctionne même sans elles.

```
"""
ÉTAPE 9 (optionnelle) — Démographie de l'audience : âge, genre, pays.

YouTube ne donne ces données QUE pour une chaîne dont on est propriétaire,
via la YouTube Analytics API, et uniquement en POURCENTAGES (jamais par personne).

Préparation (mode réel) :
  1. Google Cloud Console -> activer « YouTube Analytics API »
  2. Identifiants -> « ID client OAuth » -> type « Application de bureau »
  3. Télécharger le JSON et l'enregistrer sous client_secret.json dans ce dossier
  4. Écran de consentement OAuth : ajouter son adresse Gmail comme « utilisateur test »
  5. Lancer le script : un navigateur s'ouvre pour se connecter au compte de la chaîne

Lancement :  python 09_demographie.py --jours 90
```

```
Sans client_secret.json (ou avec YT_API_KEY=DEMO) : données simulées.
"""

import argparse
import os
import random
from datetime import date, timedelta

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import pandas as pd
from sqlalchemy import delete

from db import demographics, get_engine
```

Bloc 2/6 — La période et le mode

On analyse les `--jours` derniers jours, en s'arrêtant 2 jours avant aujourd'hui (délai des données Analytics). Le mode DEMO est choisi si la clé vaut DEMO ou si `client_secret.json` est absent.

```
parser = argparse.ArgumentParser()
parser.add_argument("--jours", type=int, default=90)
args = parser.parse_args()
fin = date.today() - timedelta(days=2)  # les données Analytics ont ~2 jours de retard
debut = fin - timedelta(days=args.jours)
os.makedirs("figures", exist_ok=True)

demo = os.getenv("YT_API_KEY", "DEMO").upper() == "DEMO" or not os.path.exists(
    "client_secret.json"
)
```

Bloc 3/6 — Mode DEMO : données simulées

On fabrique une répartition réaliste par âge et genre, puis par pays, avec un générateur aléatoire à graine fixe (`random.Random(3)`) pour obtenir toujours les mêmes chiffres.

```
if demo:
    print(
        "Mode DÉMO : démographie simulée "
        "(ajoute client_secret.json pour tes vraies données).\n"
    )
    r = random.Random(3)
    ages = ["age13-17", "age18-24", "age25-34", "age35-44", "age45-54", "age55-64", "age65-"]
    poids = [8, 34, 30, 14, 8, 4, 2]
    rows_ag = []
    for a, w in zip(ages, poids):
        h = r.uniform(0.5, 0.65)
        rows_ag += [[a, "male", w * h], [a, "female", w * (1 - h)]]
    tot = sum(x[2] for x in rows_ag)
    rows_ag = [[a, g, round(100 * x / tot, 2)] for a, g, x in rows_ag]
    pays = {"CG": 46, "CD": 18, "FR": 12, "CI": 5, "CM": 4, "BE": 4, "US": 3, "GA": 3, "AO": 2}
    rows_c = [
        [c, n * 100, n * 100 * r.uniform(2.0, 4.5), r.uniform(80, 260)]
        for c, n in pays.items()
    ]
```

Bloc 4/6 — Mode réel : YouTube Analytics

`InstalledAppFlow` ouvre le navigateur pour la connexion OAuth. `ids="channel==MINE"` signifie « ma chaîne ». La première requête demande `viewerPercentage` par âge et genre, la seconde les vues et le temps de visionnage par pays.

```
else:
    from google_auth_oauthlib.flow import InstalledAppFlow
    from googleapiclient.discovery import build

    flow = InstalledAppFlow.from_client_secrets_file(
        "client_secret.json", scopes=["https://www.googleapis.com/auth/yt-analytics.readonly"]
    )
    creds = flow.run_local_server(port=0)
    yta = build("youtubeAnalytics", "v2", credentials=creds)
    rows_ag = (
        yta.reports()
        .query(
            ids="channel==MINE",
            startDate=str(debut),
            endDate=str(fin),
            metrics="viewerPercentage",
            dimensions="ageGroup,gender",
            sort="gender,ageGroup",
        )
        .execute()
        .get("rows", [])
    )
    rows_c = (
        yta.reports()
        .query(
            ids="channel==MINE",
            startDate=str(debut),
            endDate=str(fin),
            metrics="views,estimatedMinutesWatched,averageViewDuration",
            dimensions="country",
            sort="-views",
            maxResults=25,
        )
        .execute()
        .get("rows", [])
    )
    if not rows_ag:
        print(
            "[ATTENTION] Pas de données d'âge/genre : "
            "la chaîne n'a pas encore assez de vues sur la période."
        )
```

Bloc 5/6 — Enregistrer en base

Les résultats sont mis en DataFrames puis enregistrés dans la table `demographics` (après avoir vidé les anciennes valeurs avec `delete`).

```
ag = pd.DataFrame(rows_ag, columns=["age_group", "gender", "pourcentage"])
pc = pd.DataFrame(rows_c, columns=["country", "views", "minutes", "duree_moy_s"])

with get_engine().begin() as conn:
    conn.execute(delete(demographics))
    for _, x in ag.iterrows():
        conn.execute(
            demographics.insert().values(
                dimension="age_gender",
                age_group=x.age_group,
                gender=x.gender,
                metric="viewerPercentage",
                value=float(x.pourcentage),
                start_date=str(debut),
```

```
                end_date=str(fin),
            )
        )
    for _, x in pc.iterrows():
        for m in ("views", "minutes", "duree_moy_s"):
            conn.execute(
                demographics.insert().values(
                    dimension="country",
                    country=x.country,
                    metric=m,
                    value=float(x[m]),
                    start_date=str(debut),
                    end_date=str(fin),
                )
            )
```

Bloc 6/6 — Afficher et dessiner

`pivot_table` réorganise les données en tableau croisé : une ligne par tranche d'âge, une colonne par genre. On calcule aussi la part de vues de chaque pays.

```
if not ag.empty:
    tab = ag.pivot_table(
        index="age_group", columns="gender", values="pourcentage", aggfunc="sum"
    ).fillna(0)
    print(f"Audience par âge et genre (% des vues, du {debut} au {fin}) :")
    print(tab.round(1).to_string())
    print("\nTotal par genre :", ag.groupby("gender")["pourcentage"].sum().round(1).to_dict())
    tab.plot.bar(figsize=(9, 5), title="Audience par âge et genre (%)")
    plt.tight_layout()
    plt.savefig("figures/demographie_age_genre.png", dpi=120)
    plt.close()
if not pc.empty:
    pc["part_vues_%"] = 100 * pc["views"] / pc["views"].sum()
    print("\nAudience par pays :")
    print(pc.round(1).to_string(index=False))
    print(
        "\nQuestion : les pays où l'on regarde le plus longtemps "
        "sont-ils ceux qui regardent le plus ?"
    )
print("\nDonnées enregistrées dans la table demographics, graphique dans figures/")
```

Lancer

```
python 09_demographie.py --jours 90
```

Résultat attendu (mode DEMO)

```
Mode DÉMO : démographie simulée (ajoute client_secret.json pour tes vraies données).

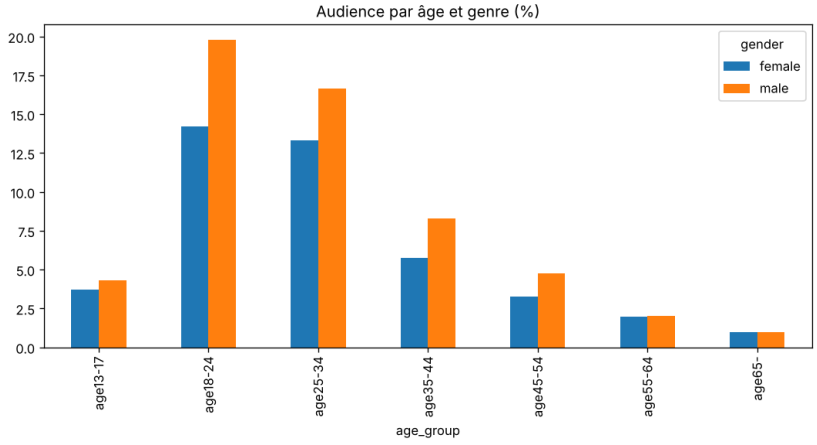
Audience par âge et genre (% des vues, du 2026-07-28 au 2026-10-26) :
gender     female  male
age_group
age13-17      3.7   4.3
age18-24     14.2  19.8
age25-34     13.3  16.7
age35-44      5.7   8.3
age45-54      3.2   4.8
age55-64      2.0   2.0
age65-        1.0   1.0

Total par genre : {'female': 43.2, 'male': 56.8}

Audience par pays :
country  views  minutes  duree_moy_s  part_vues_%
     CG   4600  18830.9        126.7         47.4
     CD   1800   4654.5        259.2         18.6
     FR   1200   3810.8        230.6         12.4
     CI    500   1595.4        195.0          5.2
     CM    400    950.6        194.3          4.1
     BE    400   1668.0        174.2          4.1
     US    300   1155.9        200.9          3.1
     GA    300    648.0        216.5          3.1
     AO    200    695.5        134.2          2.1

Question : les pays où l'on regarde le plus longtemps sont-ils ceux qui regardent le plus ?

Données enregistrées dans la table demographics, graphique dans figures/
```



figures/demographie_age_genre.png (mode DEMO).

Pour les vraies données

1. Google Cloud Console → activer **YouTube Analytics API**.
2. Identifiants → **ID client OAuth** → Application de bureau → télécharger le JSON.
3. L'enregistrer sous `client_secret.json` dans le dossier du projet.
4. Écran de consentement OAuth : ajouter son Gmail comme **utilisateur test**.
5. Lancer le script : le navigateur s'ouvre pour se connecter à la chaîne.

10 Le tableau de bord en direct

`streamlit run dashboard.py` — 25 min

Objectif : Afficher les résultats en direct dans une page web.

Fichiers à créer : `dashboard.py`

Fichier `dashboard.py` — nouveau fichier

Le tableau de bord web, construit avec Streamlit. Il relit la base à intervalle régulier et affiche indicateurs et graphiques interactifs.

Bloc 1/9 — Imports et en-tête de la page

Avec Streamlit, chaque instruction `st.quelquechose()` ajoute un élément à la page web. `layout="wide"` utilise toute la largeur de l'écran.

```
"""
ÉTAPE 10 — Tableau de bord en temps réel (Streamlit).

Lancement (dans un 2e terminal, pendant que le collecteur tourne) :
    streamlit run dashboard.py
"""

from datetime import timedelta

import pandas as pd
import plotly.express as px
import streamlit as st

from donnees import croissance, lire, releves, table_commentaires, table_videos

st.set_page_config(page_title="Data Mining YouTube — temps réel", layout="wide")
st.title("Data Mining YouTube — suivi en temps réel")
```

Bloc 2/9 — Deux petites fonctions utilitaires

Elles affichent un graphique ou un tableau sur toute la largeur, en restant compatibles avec les anciennes et les nouvelles versions de Streamlit (d'où le `try / except`).

```
def graphique(zone, fig):
    """Affiche un graphique Plotly sur toute la largeur (toutes versions)."""
    try:
        zone.plotly_chart(fig, width="stretch")
    except Exception:  # anciennes versions de Streamlit
        zone.plotly_chart(fig, use_container_width=True)


def tableau(zone, df, **kw):
    try:
        zone.dataframe(df, width="stretch", **kw)
    except Exception:  # anciennes versions de Streamlit
        zone.dataframe(df, use_container_width=True, **kw)
```

Bloc 3/9 — La barre latérale

Tout ce qui est dans `with st.sidebar:` s'affiche dans la colonne de gauche : un curseur pour la fréquence d'actualisation, une liste pour la période, une case à cocher. Leurs valeurs sont stockées dans des variables.

```
with st.sidebar:
    st.header("Réglages")
    refresh = st.slider("Actualisation (secondes)", 10, 300, 30, step=10)
    fenetre = st.selectbox("Période affichée", ["30 min", "2 h", "12 h", "Tout"], index=3)
    seulement_suivies = st.checkbox("Seulement les vidéos suivies", value=True)
    st.caption("Le collecteur (étape 3) doit tourner dans un autre terminal.")
```

Bloc 4/9 — L'actualisation automatique

Le **décorateur** `@st.fragment(run_every=...)` relance la fonction toutes les N secondes : seule cette partie de la page est redessinée. On y recharge les données et on applique les filtres choisis.

```
@st.fragment(run_every=timedelta(seconds=refresh))
def tableau_de_bord():
    v = table_videos()
    if v.empty:
        st.warning("Base vide : lance `python 02_choisir_videos.py` puis le collecteur.")
        return
    if seulement_suivies:
        v = v[v["tracked"] == 1]
    s = releves()
    s = s[s["video_id"].isin(v["video_id"])]
    if fenetre != "Tout" and not s.empty:
        h = {"30 min": 0.5, "2 h": 2, "12 h": 12}[fenetre]
        s = s[s["collected_at"] >= s["collected_at"].max() - pd.Timedelta(hours=h)]
    c = table_commentaires()
    c = c[c["video_id"].isin(v["video_id"])] if not c.empty else c

    st.caption(
        f"Dernière actualisation : {pd.Timestamp.now():%H:%M:%S} — dernier relevé : "
        f"{s['collected_at'].max() if not s.empty else '—'} (UTC)"
    )
```

Bloc 5/9 — Les indicateurs clés

`st.columns(5)` crée 5 colonnes ; dans chacune, `metric` affiche un chiffre en grand.

```
    # ---------- indicateurs ----------
    k = st.columns(5)
    k[0].metric("Vidéos suivies", len(v))
    k[1].metric("Vues totales", f"{int(v['view_count'].sum()):,}".replace(",", " "))
    k[2].metric("Relevés", len(s))
    k[3].metric("Commentaires collectés", len(c))
    if not c.empty and c["sentiment"].notna().any():
        k[4].metric(
            "Commentaires positifs", f"{100 * (c['sentiment'] == 'positif').mean():.0f} %"
        )
    else:
        k[4].metric("Commentaires positifs", "—", help="Lance l'étape 5")
```

Bloc 6/9 — Courbes et barres

`px.line` trace l'évolution des vues (une couleur par vidéo), `px.bar` la vitesse et l'engagement. Plotly rend les graphiques interactifs : survol, zoom, légende cliquable.

```
    # ---------- courbes temps réel ----------
    titres = v.set_index("video_id")["title"].str[:40]
    if not s.empty:
        s = s.assign(video=s["video_id"].map(titres))
        graphique(
            st,
            px.line(
                s,
                x="collected_at",
                y="view_count",
                color="video",
                markers=True,
                log_y=True,
                title="Évolution des vues (échelle log)",
```

```
            labels={"collected_at": "Heure (UTC)", "view_count": "Vues"},
        ),
    )
col1, col2 = st.columns(2)
g = croissance()
g = (
    g[g["video_id"].isin(v["video_id"])].dropna(subset=["vues_par_minute"])
    if not g.empty
    else g
)
if not g.empty:
    last = g.groupby("video_id").tail(1).assign(video=lambda d: d["video_id"].map(titres))
    graphique(
        col1,
        px.bar(
            last.sort_values("vues_par_minute"),
            x="vues_par_minute",
            y="video",
            orientation="h",
            title="Vitesse actuelle (vues / minute)",
        ),
    )
graphique(
    col2,
    px.bar(
        v.sort_values("taux_engagement").assign(video=lambda d: d["title"].str[:40]),
        x="taux_engagement",
        y="video",
        orientation="h",
        title="Taux d'engagement (likes + commentaires) / vues",
    ),
)
```

Bloc 7/9 — Le sentiment

Un camembert pour la répartition globale et des barres empilées par vidéo, avec des couleurs fixes : vert = positif, gris = neutre, rouge = négatif.

```
# ---------- sentiment ----------
if not c.empty and c["sentiment"].notna().any():
    col3, col4 = st.columns(2)
    graphique(
        col3,
        px.pie(
            c,
            names="sentiment",
            title="Sentiment des commentaires",
            color="sentiment",
            color_discrete_map={
                "positif": "#2e9e5b",
                "neutre": "#9aa0a6",
                "negatif": "#d0453b",
```

```
                },
            ),
        )
        par_v = (
            c.dropna(subset=["sentiment"])
            .assign(video=lambda d: d["video_id"].map(titres))
            .groupby(["video", "sentiment"])
            .size()
            .reset_index(name="nb")
        )
        graphique(
            col4,
            px.bar(
                par_v,
                x="nb",
                y="video",
                color="sentiment",
                orientation="h",
                title="Sentiment par vidéo",
                color_discrete_map={
                    "positif": "#2e9e5b",
                    "neutre": "#9aa0a6",
                    "negatif": "#d0453b",
                },
            ),
        )
```

Bloc 8/9 — Clusters et démographie

Ces graphiques n'apparaissent que si les étapes 6 et 9 ont été faites (`if v["cluster"].notna().any()`, `if not d.empty`).

```
    # ---------- clusters ----------
    if v["cluster"].notna().any():
        graphique(
            st,
            px.scatter(
                v.dropna(subset=["cluster"]).assign(
                    cluster=lambda d: d["cluster"].astype(int).astype(str)
                ),
                x="age_heures",
                y="vues_par_heure",
                color="cluster",
                hover_name="title",
                log_y=True,
                title="Clusters (étape 6) : âge vs vues par heure",
```

```
            ),
        )

    # ---------- démographie ----------
    d = lire("demographics")
    if not d.empty:
        col5, col6 = st.columns(2)
        ag = d[d["dimension"] == "age_gender"]
        graphique(
            col5,
            px.bar(
                ag,
                x="age_group",
                y="value",
                color="gender",
                barmode="group",
                title="Audience par âge et genre (%) — ta chaîne",
            ),
        )
        pc = d[(d["dimension"] == "country") & (d["metric"] == "views")]
        graphique(
            col6,
            px.bar(
                pc.sort_values("value"),
                x="value",
                y="country",
                orientation="h",
                title="Vues par pays — ta chaîne",
            ),
        )
```

Bloc 9/9 — Les derniers commentaires et le lancement

Un tableau des 30 commentaires les plus récents. La dernière ligne, `tableau_de_bord()`, appelle la fonction pour afficher la page.

```
    # ---------- derniers commentaires ----------
    if not c.empty:
        st.subheader("Derniers commentaires")
        cols = [
            x
            for x in [
                "published_at",
                "text",
                "sentiment",
                "sentiment_score",
                "is_suspect",
                "like_count",
            ]
            if x in c
        ]
        tableau(
            st, c.sort_values("published_at", ascending=False)[cols].head(30), hide_index=True
        )


tableau_de_bord()
```

Lancer

```
streamlit run dashboard.py
```

Le navigateur s'ouvre sur `http://localhost:8501`. Le collecteur doit tourner dans l'autre terminal : les graphiques bougent alors tout seuls.

Zone	Contenu
Indicateurs	vidéos suivies, vues totales, relevés, commentaires, % positifs
Évolution des vues	courbes en direct de chaque vidéo suivie
Vitesse et engagement	vues par minute actuelles, taux d'engagement
Sentiment	répartition globale et par vidéo (après l'étape 5)
Clusters, démographie	après les étapes 6 et 9
Derniers commentaires	les 30 plus récents, sentiment et indicateur « suspect »

À retenir

Arrêter le dashboard : Ctrl + C dans son terminal. Le collecteur, lui, continue de tourner tant que vous ne l'arrêtez pas.

5. Les techniques de data mining expliquées simplement

5.1 Supervisé ou non supervisé ?

	Apprentissage supervisé	Apprentissage non supervisé
Principe	On donne des exemples avec la bonne réponse	On donne des données sans réponse
Question	« Quelle est la bonne étiquette ? »	« Quels éléments se ressemblent ? »
Dans le TP	Sentiment (étape 5), prédiction (étape 7)	Clustering (étape 6), anomalies (étape 8)

5.2 TF-IDF : transformer un texte en nombres

Un algorithme ne comprend que des nombres. **TF-IDF** donne à chaque mot d'un commentaire un poids :

- **TF** (fréquence du terme) : plus un mot apparaît dans le commentaire, plus il compte.
- **IDF** (fréquence inverse dans les documents) : un mot présent partout (« le », « de ») compte peu ; un mot rare et caractéristique (« magnifique », « arnaque ») compte beaucoup.

Le TP utilise deux familles de variables : les **mots** (et paires de mots, comme « pas bon ») et des **morceaux de 2 à 5 lettres** (« magn », « nifi »...). Ces morceaux rendent le modèle robuste aux fautes d'orthographe et aux variantes (« décu », « déçu », « deçue »).

5.3 La régression logistique

Malgré son nom, c'est un algorithme de **classification**. Il apprend un poids pour chaque mot : « merci », « bravo » poussent vers *positif* ; « nul », « déçu » vers *négatif*. Pour un nouveau commentaire, il additionne les poids et en déduit une **probabilité** pour chaque classe. Avantage pédagogique : on peut afficher les mots les plus influents et comprendre ses décisions.

5.4 Évaluer un modèle

La validation croisée

On découpe le corpus en 5 parts. On entraîne sur 4 parts et on teste sur la 5e, puis on recommence 5 fois en changeant la part de test. Chaque phrase est ainsi testée une fois par un modèle qui ne l'a jamais vue.



Validation croisée à 5 plis.

Les mesures

Mesure	Question à laquelle elle répond	Exemple pour « positif »
Exactitude (accuracy)	Quelle part des prédictions est correcte ?	76 % des commentaires bien classés
Précision	Quand le modèle dit « positif », a-t-il raison ?	80 % des « positifs » prédits le sont vraiment
Rappel	Parmi les vrais positifs, combien sont retrouvés ?	70 % des vrais positifs détectés
F1	Compromis entre précision et rappel	0,75
Matrice de confusion	Quelles classes sont confondues entre elles ?	positif souvent pris pour négatif

5.5 La standardisation

Avant K-Means, chaque variable est ramenée à une moyenne de 0 et un écart-type de 1. Sans cela, la vitesse (des milliers de vues par minute) écraserait le taux d'engagement (environ 0,05) : l'algorithme ne regarderait presque qu'une seule variable.

5.6 K-Means

K-Means cherche k groupes. Il place k centres au hasard, rattache chaque vidéo au centre le plus proche, déplace chaque centre au milieu de son groupe, et recommence jusqu'à ce que plus rien ne bouge.

- **Inertie** (méthode du coude) : somme des distances au centre. Elle baisse toujours quand k augmente ; on cherche le « coude » où elle cesse de baisser fortement.
- **Silhouette** (entre −1 et 1) : chaque vidéo est-elle plus proche de son groupe que des autres ? Plus c'est haut, mieux les groupes sont séparés.

- **ACP** (analyse en composantes principales) : résume les 6 variables en 2 axes pour dessiner les groupes sur un plan.

5.7 Random Forest

Une **forêt aléatoire** combine des centaines d'**arbres de décision**. Chaque arbre pose une suite de questions (« plus de 100 000 abonnés ? », « publiée il y a plus de 24 h ? ») et donne une estimation. La forêt fait la moyenne de tous les arbres, ce qui la rend plus fiable qu'un arbre seul. Elle indique aussi l'**importance** de chaque variable.

Le R² (coefficient de détermination)

- **R² = 1** : prédictions parfaites.
- **R² = 0** : pas mieux que prédire toujours la moyenne.
- **R² négatif** : pire que la moyenne.

On prédit **log10(vues)** plutôt que les vues : se tromper de 10 000 vues est grave pour une vidéo à 5 000 vues mais négligeable pour une vidéo à 2 millions. Le logarithme mesure l'erreur en « facteur » (×2, ×5) plutôt qu'en nombre absolu.

5.8 Le z-score robuste

Le z-score mesure à combien d'« écarts habituels » une valeur se trouve de la normale :

```
z = (valeur − médiane) / MAD        MAD = écart absolu médian
```

On utilise la **médiane** et non la moyenne, car la moyenne serait elle-même tirée vers le haut par le pic qu'on cherche à détecter. Un z supérieur à 5 signifie « extrêmement inhabituel pour cette vidéo ».

5.9 Isolation Forest

L'idée : une valeur anormale est **facile à isoler**. L'algorithme découpe les données au hasard, encore et encore. Un spammeur (beaucoup de liens, compte récent, messages répétés) se retrouve seul après très peu de découpes ; un utilisateur ordinaire, noyé dans la masse, en demande beaucoup plus. Moins il faut de découpes, plus le score d'anomalie est élevé.

6. Les variables : lesquelles, d'où, pourquoi

6.1 Variables collectées

Variable	Source	Temps réel ?	Utilité
Vues, likes, nombre de commentaires	videos.list	Oui, chaque minute	Croissance, engagement
Titre, description, tags	videos.list	Non (une fois)	Analyse de texte, prédiction
Catégorie, langue	videos.list	Non	Comparaisons
Date de publication, durée	videos.list	Non	Âge, heure, prédiction
Abonnés, vidéos, pays de la chaîne	channels.list	Mis à jour régulièrement	Prédiction, viralité
Texte et likes des commentaires	commentThreads.list	Toutes les 10 min	Sentiment, spam
Ancienneté, pays déclaré du commentateur	channels.list	À la découverte de l'auteur	Anomalies
Âge, genre, pays de l'audience	YouTube Analytics	Non (1 à 2 jours de retard)	Démographie

6.2 Variables calculées (fichier donnees.py)

Variable	Formule	Interprétation
taux_engagement	(likes + commentaires) / vues	Implication du public, indépendamment de la taille
vues_par_heure	vues / âge en heures	Popularité à âge égal
vues_par_minute	nouvelles vues / minutes entre deux relevés	Vitesse instantanée (temps réel)
ratio_vues_abonnes	vues / abonnés	Supérieur à 1 : la vidéo dépasse le public habituel
age_heures	heure du relevé − heure de publication	Récente ou ancienne
heure_publication, jour	extraits de la date	Meilleur moment pour publier
nb_tags, longueur_titre	comptages	Effet de l'optimisation du titre
sentiment, sentiment_score	modèle de l'étape 5	Ton des commentaires et confiance du modèle

6.3 Ce qui n'est PAS disponible

Attention

YouTube ne donne **jamais** l'âge, le genre, la ville ou la nationalité d'un spectateur ou d'un commentateur. Seul le propriétaire d'une chaîne voit ces informations, en pourcentages.

Le « pays » d'un commentateur n'est connu que s'il l'a déclaré sur sa chaîne, ce qui est rare : dans la démo, plus de la moitié ne déclarent rien. Cette variable est donc incomplète et biaisée.

7. Éthique et protection des données

Travailler sur des données de réseaux sociaux impose des règles. Elles font partie de la compétence d'un data scientist, et du barème du TP.

Principe	Application dans le TP
Donnée publique ≠ donnée libre	Un commentaire public reste une donnée personnelle de son auteur.
Minimisation	On ne collecte que ce qui sert l'analyse : pas de nom, pas de photo de profil.
Anonymisation dès la collecte	Les identifiants sont remplacés par un hash salé avant d'être stockés.
Analyse de groupe	On publie des statistiques (« 30 % des commentateurs... »), jamais un profil individuel.
Pas de déduction sensible	Interdit de deviner le genre, l'âge ou l'origine à partir d'un nom ou d'une photo.
Respect de la plateforme	API officielle, quota respecté, pas de republication des données brutes.
Cadre légal	Au Congo, la loi de 2019 sur la protection des données personnelles s'applique.

8. Problèmes fréquents et solutions

Message ou symptôme	Cause probable	Solution
ModuleNotFoundError	Environnement virtuel non activé	.venv\Scripts\activate puis pip install -r requirements.txt
Access denied for user 'root'	Mauvais mot de passe MySQL	Corriger MYSQL_PASSWORD dans .env
Can't connect to MySQL server	MySQL n'est pas démarré	XAMPP : bouton Start à côté de MySQL
400 keyInvalid	Clé mal copiée	Recopier la clé sans espace
403 accessNotConfigured	API non activée	Activer YouTube Data API v3 dans la console
403 quotaExceeded	Quota du jour épuisé	Attendre le lendemain matin (8 h ou 9 h à Brazzaville) ou passer en DEMO
Aucune tendance pour CG	Tendances indisponibles pour ce pays	--region FR, --region CD ou --search
« Il faut au moins 4 relevés »	Collecte trop courte	Laisser tourner le collecteur 5 minutes de plus
Le dashboard ne bouge pas	Le collecteur est arrêté	Le relancer dans le premier terminal
Repartir de zéro (DEMO)	—	Vider la base et supprimer .demo_state.json

9. Glossaire

Terme	Définition
API	Interface qui permet à un programme d'obtenir des données d'un service.
Apprentissage supervisé	Le modèle apprend à partir d'exemples dont on connaît la bonne réponse.
Apprentissage non supervisé	Le modèle cherche une structure dans des données sans réponse connue.
Clé API	Code secret qui identifie votre projet auprès de Google.
Cluster	Groupe d'éléments qui se ressemblent.
Corpus	Ensemble de textes utilisé pour entraîner ou évaluer un modèle.
DataFrame	Tableau de données manipulé avec pandas (lignes et colonnes).
Dédoublonnage	Vérifier qu'une donnée n'est pas enregistrée deux fois.
Engagement	Réaction du public : likes et commentaires rapportés aux vues.
Faux positif	Alerte déclenchée à tort (un fan actif pris pour un spammeur).
Fuite de données	Utiliser pour prédire une information qu'on n'aurait pas encore en situation réelle.
Hash	Empreinte irréversible d'un texte, utilisée pour anonymiser.
JSON	Format texte de données structurées en paires clé : valeur.
OAuth	Méthode de connexion sécurisée à un compte Google sans donner son mot de passe au programme.
Quota	Nombre maximum d'unités de requêtes autorisées par jour.
Série temporelle	Suite de mesures d'une même grandeur au fil du temps.
Standardisation	Mise à l'échelle des variables (moyenne 0, écart-type 1).
TF-IDF	Méthode qui transforme un texte en nombres en pondérant les mots selon leur rareté.
UTC	Temps universel coordonné. Brazzaville est à UTC+1.
Validation croisée	Évaluation d'un modèle en le testant successivement sur différentes parts des données.

Annexe A — Le simulateur (simulator.py)

Ce fichier est nécessaire pour travailler en mode DEMO (sans clé API ou sans Internet). Créez-le à l'étape 1, dans le dossier du projet, à côté de youtube_client.py.

Fichier `simulator.py` — à copier tel quel

Le simulateur de l'API YouTube, utilisé quand YT_API_KEY=DEMO. Il n'est pas nécessaire de le comprendre en détail : copiez-le tel quel. Il renvoie des réponses au même format JSON que YouTube, avec des compteurs qui augmentent vraiment au fil du temps.

Bloc 1/9 — Paramètres généraux

STATE_FILE mémorise l'heure de démarrage de la simulation, pour que tous les scripts voient les mêmes vidéos au même stade.

```
"""
simulator.py — Simulateur de l'API YouTube (mode DEMO, hors ligne).

Il renvoie des réponses JSON au MÊME FORMAT que la vraie API, avec des
compteurs qui augmentent réellement avec le temps. Idéal pour :
  - les salles sans Internet,
  - tester le pipeline avant d'avoir une clé API.

Les vidéos ont des profils de croissance différents (explosive, lente,
démarrage tardif, déjà stable) et une vidéo contient une anomalie
(hausse brutale de vues) pour l'exercice de détection d'anomalies.
"""

import hashlib
```

```
import json
import math
import os
import random
import time
from datetime import datetime, timedelta, timezone

STATE_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), ".demo_state.json")
N_VIDEOS = 40
N_CHANNELS = 12
N_USERS = 400
N_SPAMMERS = 10
```

Bloc 2/9 — Titres, catégories et tags

Le vocabulaire utilisé pour inventer les vidéos.

```
TITRES = [
    "Rumba congolaise — nouveau clip officiel",
    "Ndombolo 2026 : la danse qui enflamme Brazzaville",
    "Diables Rouges : résumé du match",
    "Sketch : quand ta maman découvre ton bulletin",
    "Pointe-Noire by night",
    "Recette du saka-saka de grand-mère",
    "Comment réussir son bac en 3 mois",
    "Débat : la jeunesse et le chômage",
    "Freestyle rap kongo",
    "Micro-trottoir au marché Total",
    "Les meilleurs buts de la semaine",
    "Mariage coutumier à Brazzaville",
    "Comédie : le taxi-bus",
```

```
    "Tutoriel : créer son entreprise au Congo",
    "Concert live au Palais des congrès",
    "Top 10 des plats congolais",
    "Actualité : ce qui a changé cette semaine",
    "Prank au quartier Bacongo",
    "Interview exclusive d'un artiste",
    "Vlog : une journée à Oyo",
]
CATS = ["10", "10", "10", "23", "23", "24", "24", "17", "25", "27", "22", "26"]
TAGS = [
    "congo",
    "brazzaville",
    "rumba",
    "ndombolo",
    "humour",
    "musique",
    "football",
    "afrique",
    "kinshasa",
    "pointe-noire",
    "clip",
    "buzz",
    "2026",
    "lingala",
    "comédie",
]
```

Bloc 3/9 — Les textes de commentaires

Des commentaires positifs, négatifs, neutres et des messages de spam.

```
POSITIFS = [
    "Trop fort mon frère, continue comme ça !",
    "Merci mingi pour cette vidéo",
    "Ezali malamu, j'adore",
    "Le clip est magnifique, bravo à toute l'équipe",
    "Je regarde ça en boucle depuis ce matin",
    "Fierté congolaise !",
    "Super contenu, très instructif",
    "Tu m'as fait trop rire hahaha",
    "Quelle voix, c'est incroyable",
    "Meilleure vidéo de la semaine",
    "Bravo, vous représentez bien le pays",
    "J'aime beaucoup, abonné direct",
    "C'est du lourd ça !",
```

```
    "Merci pour les conseils, ça aide vraiment",
    "La danse est trop belle",
    "Respect à toi l'artiste",
    "Excellent travail, très professionnel",
]
NEGATIFS = [
    "C'est nul, je ne comprends pas le buzz",
    "Ezali mabe, déçu",
    "Vidéo trop longue et ennuyeuse",
    "Le son est mauvais, on n'entend rien",
    "Arrêtez de copier les autres artistes",
    "Pas terrible cette fois-ci",
    "Quelle perte de temps",
    "Je n'aime pas du tout ce style",
    "Le titre est trompeur",
    "Trop de publicités, c'est fatigant",
    "Franchement c'est raté",
    "Mauvaise qualité d'image",
    "Ce n'est pas drôle du tout",
    "Encore la même chose, aucune créativité",
]
NEUTRES = [
    "Mbote na bino",
    "Qui regarde en 2026 ?",
    "C'est tourné où exactement ?",
    "Première fois que je vois ça",
    "Le titre de la musique à 2:15 svp ?",
    "Je suis de Pointe-Noire",
    "Quelqu'un de Kinshasa ici ?",
    "On attend la suite",
    "C'est sorti quand ?",
    "Je partage avec mes amis",
    "Présent depuis Dolisie",
    "Quel est le nom du danseur ?",
    "Vu à la télé hier soir",
]
SPAMS = [
    "Abonnez-vous à ma chaîne !!! http://bit.ly/gagner-argent",
    "GAGNEZ 500 000 FCFA PAR JOUR, contactez-moi WhatsApp",
    "Cliquez ici pour des vues gratuites http://vues-gratuites.xyz",
    "Abonne-toi à moi je m'abonne à toi !!!",
    "Promo crypto : doublez votre argent maintenant http://crypto-promo.top",
]
```

■ Bloc 4/9 — Hasard reproductible et dates

_rng crée un générateur aléatoire à partir d'un texte : la même vidéo reçoit toujours les mêmes caractéristiques.

```
def _rng(*keys) -> random.Random:
    h = hashlib.md5("|".join(map(str, keys)).encode()).hexdigest()
    return random.Random(int(h, 16))


def _start_time() -> float:
    if os.path.exists(STATE_FILE):
        with open(STATE_FILE) as f:
            return json.load(f)["start"]
    start = time.time()
    with open(STATE_FILE, "w") as f:
        json.dump({"start": start}, f)
    return start


def _iso(ts: float) -> str:
    return datetime.fromtimestamp(ts, tz=timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")
```

■ Bloc 5/9 — Les caractéristiques d'une vidéo

Chaque vidéo reçoit un profil (explosive, lente, tardive, stable), un plafond de vues, des taux de likes et de commentaires. La vidéo n° 7 est l'anomalie.

```python
def _video_params(i: int) -> dict:
    r = _rng("video", i)
    profil = r.choices(["explosive", "lente", "tardive", "stable"], [3, 3, 2, 2])[0]
    p = {
        "id": f"demoVid{i:03d}",
        "channel": f"demoChan{r.randrange(N_CHANNELS):02d}",
        "title": f"{r.choice(TITRES)} #{i}",
        "category": r.choice(CATS),
        "tags": r.sample(TAGS, r.randint(0, 8)),
        "duration": r.choice([45, 90, 180, 240, 300, 420, 600, 900, 1500]) + r.randint(0, 59),
        "profil": profil,
        "like_rate": r.uniform(0.02, 0.09),
        "comment_rate": r.uniform(0.002, 0.012),
        "humeur": r.uniform(-0.4, 0.6),  # tendance des commentaires (négatif -> positif)
    }
    if profil == "explosive":
        p.update(
            offset_min=r.uniform(5, 40), cap=r.uniform(80_000, 400_000), tau=r.uniform(20, 60)
        )
    elif profil == "lente":
        p.update(
            offset_min=r.uniform(10, 120),
            cap=r.uniform(10_000, 60_000),
            tau=r.uniform(200, 600),
        )
    elif profil == "tardive":
        p.update(
            offset_min=r.uniform(5, 30),
            cap=r.uniform(50_000, 250_000),
            t0=r.uniform(40, 120),
            s=r.uniform(8, 20),
        )
    else:  # stable : publiée il y a plusieurs jours, déjà saturée
        p.update(
            offset_min=r.uniform(2 * 1440, 10 * 1440),
            cap=r.uniform(100_000, 2_000_000),
            tau=r.uniform(300, 900),
        )
    p["anomalie"] = i == 7  # cette vidéo reçoit des "vues achetées"
    if p["anomalie"]:  # vidéo très vue (donc toujours suivie) mais stable
        p.update(profil="stable", offset_min=4 * 1440, cap=1_500_000, tau=600)
    return p
```

■ Bloc 6/9 — Les compteurs à un instant donné

Croissance exponentielle vers un plafond, ou en S (courbe logistique) pour les démarrages tardifs. Après 15 minutes, la vidéo anomalie reçoit 60 000 vues d'un coup.

```python
def _counts(p: dict, now: float) -> tuple[int, int, int, float]:
    start = _start_time()
    published = start - p["offset_min"] * 60
    age_min = max(0.0, (now - published) / 60)
    if p["profil"] == "tardive":
        v = p["cap"] / (1 + math.exp(-(age_min - p["t0"]) / p["s"]))
    else:
        v = p["cap"] * (1 - math.exp(-age_min / p["tau"]))
    if p["anomalie"] and now - start > 15 * 60:
        v += 60_000  # saut brutal après 15 minutes de collecte
    v = int(v) + 50
    return v, int(v * p["like_rate"]), int(v * p["comment_rate"]), published
```

■ Bloc 7/9 — Fabriquer les réponses JSON : vidéos et chaînes

Ces fonctions construisent des dictionnaires identiques à ceux de YouTube. Les comptes de spam sont récents, sans abonnés et sans vidéos.

```
def _video_item(i: int, now: float) -> dict:
    p = _video_params(i)
    views, likes, ncom, published = _counts(p, now)
    return {
        "kind": "youtube#video",
        "id": p["id"],
        "snippet": {
            "publishedAt": _iso(published),
            "channelId": p["channel"],
            "title": p["title"],
            "description": f"Vidéo de démonstration ({p['profil']}).",
            "tags": p["tags"],
            "categoryId": p["category"],
            "defaultAudioLanguage": "fr",
        },
        "contentDetails": {"duration": f"PT{p['duration'] // 60}M{p['duration'] % 60}S"},
        "statistics": {
            "viewCount": str(views),
            "likeCount": str(likes),
            "commentCount": str(ncom),
        },
    }


def _channel_item(cid: str) -> dict:
    r = _rng("chan", cid)
    if cid.startswith("demoUser"):
        k = int(cid.replace("demoUser", ""))
        spam = k < N_SPAMMERS
        created = time.time() - (r.uniform(1, 20) if spam else r.uniform(60, 4000)) * 86400
        country = (
            None
            if spam
            else r.choices(["CG", "CD", "FR", "CI", "CM", None], [3, 2, 1, 1, 1, 8])[0]
        )
        subs = 0 if spam else int(r.paretovariate(1.2) * 3) - 3
        vids = 0 if spam else max(0, int(r.paretovariate(1.5)) - 1)
        title = f"Utilisateur {k}"
    else:
        created = time.time() - r.uniform(300, 3000) * 86400
        country = r.choice(["CG", "CG", "CD", "FR", None])
        subs = int(r.uniform(2_000, 900_000))
        vids = r.randint(20, 800)
        title = f"Chaîne Démo {cid[-2:]}"
    return {
        "kind": "youtube#channel",
        "id": cid,
        "snippet": {
            "title": title,
            "publishedAt": _iso(created),
            **({"country": country} if country else {}),
        },
        "statistics": {
            "subscriberCount": str(max(0, subs)),
            "videoCount": str(vids),
            "viewCount": str(max(0, subs) * 40),
            "hiddenSubscriberCount": False,
        },
    }
```

■ Bloc 8/9 — Fabriquer un commentaire

6 % des commentaires sont du spam.

```
def _comment_item(p: dict, idx: int, ncom: int, published: float, now: float) -> dict:
    r = _rng("com", p["id"], idx)
    if r.random() < 0.06:
        author = f"demoUser{r.randrange(N_SPAMMERS)}"
        text = r.choice(SPAMS)
    else:
        author = f"demoUser{r.randrange(N_SPAMMERS, N_USERS)}"
        x = r.random() + p["humeur"] * 0.4
        text = r.choice(NEGATIFS if x < 0.3 else NEUTRES if x < 0.55 else POSITIFS)
    t = published + (now - published) * (idx + 1) / max(ncom, 1)
    return {
        "kind": "youtube#commentThread",
        "id": f"{p['id']}-c{idx}",
        "snippet": {
            "videoId": p["id"],
            "totalReplyCount": r.choice([0, 0, 0, 1, 2, 5]),
            "topLevelComment": {
                "id": f"{p['id']}-c{idx}",
                "snippet": {
                    "textOriginal": text,
                    "authorChannelId": {"value": author},
                    "likeCount": int(r.paretovariate(1.3)) - 1,
                    "publishedAt": _iso(t),
                },
            },
        },
    }
```

■ Bloc 9/9 — Répondre à une requête

Le point d'entrée appelé par `youtube_client.py` : selon l'adresse demandée (videos, search, channels, commentThreads), il renvoie la bonne réponse.

```
def handle(endpoint: str, params: dict) -> dict:
    now = time.time()
    if endpoint == "videos":
        if params.get("chart") == "mostPopular":
            items = [_video_item(i, now) for i in range(N_VIDEOS)]
            items.sort(key=lambda it: -int(it["statistics"]["viewCount"]))
            return {"items": items[: int(params.get("maxResults", 50))]}
        ids = params.get("id", "").split(",")
        idx = [int(x.replace("demoVid", "")) for x in ids if x.startswith("demoVid")]
        return {"items": [_video_item(i, now) for i in idx if i < N_VIDEOS]}
    if endpoint == "search":
        q = params.get("q", "").lower()
        hits = [
            i
```

```
            for i in range(N_VIDEOS)
            if any(w in _video_params(i)["title"].lower() for w in q.split())
        ]
        hits = hits or list(range(N_VIDEOS))
        return {
            "items": [
                {"id": {"videoId": f"demoVid{i:03d}"}}
                for i in hits[: int(params.get("maxResults", 25))]
            ]
        }
    if endpoint == "channels":
        return {"items": [_channel_item(c) for c in params.get("id", "").split(",") if c]}
    if endpoint == "commentThreads":
        vid = params["videoId"]
        i = int(vid.replace("demoVid", ""))
        p = _video_params(i)
        _, _, ncom, published = _counts(p, now)
        n = min(int(params.get("maxResults", 100)), ncom)
        return {
            "items": [
                _comment_item(p, k, ncom, published, now)
                for k in range(ncom - 1, ncom - 1 - n, -1)
            ]
        }
    raise ValueError(f"Endpoint non simulé : {endpoint}")
```